

Report Check

24L-5066 FBA Final Project.docx

 Quick Submit Quick Submit Turnitin

Document Details

Submission ID

trn:oid:::1:7171974105

40 Pages

Submission Date

May 10, 2026, 8:51 AM

13,422 Words

93,958 Characters

Download Date

May 10, 2026, 8:52 AM

File Name

24L-5066 FBA Final Project.docx

File Size

2.13 MB

*% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **0 AI-generated only 0%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



2026

Advanced Business Intelligence and Predictive Analytics for Amazon India E-Commerce:

Customer Churn, Inventory Optimization, and Pricing Strategy Analysis

Submitted To:
Miss Aiza Aftab

Submission Date:
May 10th, 2026

Section
BSBA 4C

Presented by

Adnan
Hussain
24L-5066

Arshad
Jalil
24L-5143

Arshad
Jalil
24L-5050

Adnan
Hussain
24L-5123

**Syed Murtaza
Naqvi**
24L-5089

Table of Contents

1. Report Abstract	3
2. Introduction and Problem Statement	5
2.1 Background and Context	5
2.2 Problem Statement	5
2.3 Research Objectives	6
3. Data Source and Methodology	7
3.1 Data Source and Collection	7
3.2 Analytical Methodology	8
3.3 Scope and Limitations	9
4. Data Cleaning and Wrangling	11
4.1 Data Quality Assessment	11
4.2 Outlier Analysis and Treatment	12
4.3 Data Transformations and Feature Engineering	12
Step 1 — Type Corrections	12
Step 2 — Duplicate Removal	13
Step 3 — Feature Engineering	13
5. Exploratory Data Analysis (EDA)	14
5.1 Customer Behaviour & Order Pattern Analysis	14
5.2 Performance & Pricing Distribution Analysis	16
5.3 Correlation Analysis	18
5.4 Variable Relationships & Cross-Tabulations	20
6.1 Descriptive Statistics	22
6.2 Trend Analysis	23
6.3 Distribution Analysis	25
6.4 Comparative Analysis	25
7. Clustering Analysis (K-Means)	26
7.1 Determination of Optimal Clusters — The Elbow Method	26
7.2 Cluster Characteristics — Customer Segmentation	28
7.3 Cluster Interpretation — What the Segments Mean	31
7.4 Cluster Characteristics — Product Pricing Segmentation	32
8. Regression Analysis	35
8.1 Multiple Linear Regression Model	35
8.2 Regularised Regression — Ridge and Lasso	35
8.3 Model Diagnostics and Coefficient Interpretation	36

Amazon E-Commerce Analytics Report

9. Time Series Forecasting (ARIMA)	38
9.1 Trend-Based Decomposition	38
9.2 ARIMA Model Specification and Estimation	39
9.3 Forecast Generation and Future Demand Estimates	40
10. Optimization Analysis	42
10.1 Problem Formulation	42
10.2 Key Parameters	42
10.3 Optimal Conditions	43
10.4 Sensitivity Analysis	44
11. Monte Carlo Simulation	46
11.1 Simulation Design	46
11.2 Risk Metrics Across Strategies	46
11.3 Value at Risk (5th Percentile Analysis)	47
11.4 Interpretation and Strategic Implications	47
12. Business Insights and Recommendations	49
12.1 Recency as the Key Driver of Customer Churn	49
12.2 Pricing Strategy Differentiation by Customer Segment	49
12.3 Inventory Discipline Anchored on Forecast	50
12.4 The Role of Customer Segmentation in Personalisation	50
12.5 Operational Quality and Delivery Performance	50
12.6 Action Plan: Master Recommendations Table	51
13. Conclusion	52
14. References and Appendix	54
14.1 Dataset	54
14.2 Python Libraries Used	54
14.3 Methodological References	54
14.4 Appendix A: Code Module Map	55
14.5 Appendix B: Glossary of Technical Terms	55

1. Report Abstract

The following report constitutes a comprehensive analytical investigation into an Amazon India e-commerce dataset, which contains 10,000 customer transaction orders, generated over a 24-month period between March 2024 and March 2026. This transaction dataset is broken down into five product categories – namely Beauty, Clothing, Electronics, Home, Sports – within the five leading metropolitan markets within India - Delhi, Bangalore, Chennai, Hyderabad, and Mumbai. The purpose of this project is to transform raw transactional data into actionable business intelligence, focusing on three interdependent problem areas with direct impact on the long-term commercial success of the business – customer retention and churn, inventory and supply chain optimisation, and price and profit.

The approach followed a clear pipeline. The dataset was ingested and initial first look quality checks performed, data was then cleaned (although in actual fact the data contained 0 missing values, and 0 duplicates), data was augmented with 5 or 6 engineered features, such as, profit margin, discount % and various components of date. On exploratory data analysis it could be seen that there are almost equal distributions in terms of product categories, cities, payment types and devices, implying a well established and balanced customer base. In the first core stream of analysis an RFM framework along with K-Means was used to group the 9,942 unique customers into 4 segments- Champion, Loyal Customer, At Risk, and Lost / Churned, which were then used to train classification models to predict customer churn. Both the logistic regression and random forest classifier produced very high accuracy in predicting churn, at 99.7% and 100% respectively on holdout data, with recency being a far superior feature for predicting churn (0.9954 Importance Score).

The second column analyzed inventory. Time-series of weekly and daily demand series were summed, then decomposed to reveal trend, season, and residuals which were modeled using ARIMA(1,1,1). The forecast showed static forward demand of roughly 99 weekly orders over the following eight weeks. Optimization calculation of safety stock at 95% service level revealed a reorder point of 54 and a safety stock buffer of 11 units- operating values that can be immediately plugged into purchasing systems. The third column focused on pricing. All three regression methods tested (Linear, Ridge, and Lasso) resulted in a very stable R of about 0.9729, and showed that final sale price was strongly influenced by price itself (13,972 standardized). A K-Means clustering of products identified 3 pricing segments, "Price Sensitive", "Value Seekers", and "Premium Buyers",

Amazon E-Commerce Analytics Report

and an estimation of three discount strategies via 5,000-iteration Monte Carlo shows a "sweet spot" at a 15% discount maximizing the expected sale value (mean revenue 1.38 million dollars per simulated week) at a variance similar to that of no-discount..

Combined, all results create one distinct strategic narrative. Recency is the only immediately actionable retention tool that the company should utilize with a weekly churn score mechanism, targeting the At-Risk group (which is 4,437 customers and has a 59.9% churn rate) with tailored

Amazon E-Commerce Analytics Report

re-engagement emails. Inventory should have a reorder point of 54 units and incorporate seasonally adjusted lead times a few weeks ahead of demand periods. Pricing should be tiered: between 20-30% discounts for price-sensitive goods, between 10-15% discounts for value shoppers and low discounts on the premium product (because perceived quality suffers when goods are cheap). Section 12 lists each finding and turns each finding into a specific operational action.

2. Introduction and Problem Statement

2.1 Background and Context

The global e-commerce business has evolved into one of the richest data repositories. Each click, scroll, search, abandonment, purchase, return, review produce a structured trace; a series of which when combined and analysed illustrate consumer behavior patterns in the face of true economic incentive. At the scale of Amazon India, even small analytical insights correspond directly to quantifiable gains in revenue, inventory turns, customer lifetime value, operational costs etc. The task now isn't to collect data but to translate data into decisions.

This project falls squarely within the translation problem space. The dataset here analyzes 10,000 transactions from an Amazon India dataset spanning 31/03/2024 - 31/03/2026. Each line represents an individual purchase, detailing 20 attributes that cover all aspects of the purchase transaction: product (price, discount, final price, rating, review count, stock availability, category, subcategory, brand), customer (user_id without identifiable information, location, device type, payment mode) and fulfillment status (time in transit, delivery status, return indicator). The value of the dataset lies in this combined view: product, customer, fulfillment, and its utility in cross functional analysis.

The importance of analytics in this domain is hard to overstate. Operational efficiency is very thin in e-commerce, and small efficiency gains are compounding-one percentage point decrease in churn might retrieve thousands of customers on a large scale; one percentage point increase in preventing out-of-stocks not only affects lost sales and a secondary penalty of angry customers, but also, one percentage point increase in prevention has some fraction of impact; rigorous discount policy might increase real margin by a couple percentage points without affecting sales. All three of the problem statements discussed in this report contribute to optimizing one compounding lever.

2.2 Problem Statement

The data was analyzed using three concrete business problems stated as questions, the answer to which would lead to actual operational changes:

- **Problem 1 — Customer Retention & Churn:** How can Amazon identify patterns in customer behavior to forecast churn and develop tactics for customer retention and lifetime value enhancement? In other words, can it be identified (before a customer disappears) that they are about to disappear?

Amazon E-Commerce Analytics Report

- **Problem 2 — Inventory & Supply Chain Optimisation:** What should be Amazon's strategy in optimising its inventory management by more accurate demand forecasting to decrease out of stocks and overstocking without decreasing customer service levels? The issue is to balance two conflicting costs: the cost of carrying too much stock against the cost of carrying too little stock.

Amazon E-Commerce Analytics Report

- **Problem 3 — Pricing & Profitability:** How will Amazon use analytics to establish appropriate pricing strategies considering competitiveness, profit, and customer satisfaction at different categories of products. Is it beneficial to have a one size fit all discount policy or should segmentation be explored?

These three problems are not mutually exclusive but, in fact, synergistic. Retention fuels the customer base size. Inventory fuels the platform's capacity to satisfy that base's desires. Pricing drives the margin on satisfied transactions. Lack of strength in any one of the three will negatively affect the others and in practice these three are intertwined: very low prices will fuel near-term volume while eroding margin and stock-out likelihood; very high quality inventory management will be of no value if a customers quickly churns after first transaction; even the best churn prediction model will achieve nothing unless the intervention which it suggests is tied to the appropriate pricing offer.

2.3 Research Objectives

This analysis was divided into six distinct objectives. Each assigned specific code outputs, which are elaborated upon and interpreted in subsequent sections of this report:

1. Create a clean, validated and consistent data set for modelling downstream, applying correct data type transformations, removing duplicates, and carrying out feature engineering.
2. Profile the customer base, product inventory, and the platform operations through a detailed EDA that explores univariate, bivariate and temporal aspects.
3. Create a Recency-Frequency-Monetary (RFM) representation of each individual customer and use unsupervised K-means clustering to extract behavioural segments, after which train supervised classification models on the customer data to predict customer churn at an individual level.
4. Transform the transactional data into time series format, split it into structural components, predict forward demand using ARIMA, and translate the forecast into operational inventory thresholds via classical safety-stock optimization.
5. Develop regression models (Linear, Ridge, Lasso) to explain the final selling price based on pricing, seller and product features. Use the resulting coefficients to understand which factors drive the final price in reality.
6. Use Monte Carlo simulation to evaluate the risk-return profile of three candidate discount strategies under uncertainty, and produce a defensible recommendation that balances expected revenue against downside risk.

Each objective is fulfilled by a specific section of this report. Objective 1 corresponds to Section 4. Objective 2 to Sections 5 and 6. Objective 3 to Section 7 (clustering and the classification

Amazon E-Commerce Analytics Report

component). Objective 4 to Sections 9 and 10. Objective 5 to Section 8. Objective 6 to Section 11. The final consolidated recommendations are presented in Section 12.

3. Data Source and Methodology

3.1 Data Source and Collection

The data file (amazon10000.csv) contains 10,000 individual transactions, with 20 initial columns. Its size is roughly 1.34MB and was read through pandas read_csv function with default settings. Each line corresponds to a single purchase through Amazon India within the period observed. It's not a production database extract therefore designed for analytic purposes, and as such closely represents what a real platform might output.

The twenty columns are logically divided into 4 logical families. The first family is, identify the customer user_id(anonymised). Second, identify the product (product_id, category, subcategory, brand, price, discount, final_price, rating, review_count, stock). Third, understand seller and fulfillment details (seller_id, seller_rating, purchase_date, shipping_time_days, location, device, payment_method). Fourth, determine the transaction result is_returned (Boolean), delivery_status (categorical with four levels).

Here's a brief definition of each column and its range of values, obtained from the describe() command in Section 2:

Column	Type	Description / Observed Range
user_id	string	Anonymised customer identifier (e.g. U907645). 9,942 unique values across 10,000 rows.
product_id	string	Anonymised product identifier (e.g. P13100). 9,446 unique values.
category	categorical	Five values: Beauty, Clothing, Electronics, Home, Sports.
subcategory	categorical	Granular subdivision (e.g. Cycling, Headphones, Makeup, Kitchen).
brand	categorical	Brand of the product (Nike, Samsung, Lenovo, H&M, etc.).
price	numeric (₹)	Listed price; range ₹200–₹79,937; mean ₹13,099; median ₹4,896.
discount	numeric	Discount value as recorded in the source (range 5.01–69.99).
final_price	numeric (₹)	Price actually paid; range ₹115–₹74,699; mean ₹9,835.
rating	numeric	Customer rating 1–5; range 2.10–5.00; mean 3.92.
review_count	integer	Number of reviews on the product; range 0–1,183; mean 31.85.
stock	integer	Stock units available; range 0–500; mean 250.

Amazon E-Commerce Analytics Report

Column	Type	Description / Observed Range
seller_id	string	Anonymised seller identifier (e.g. S4473).
seller_rating	numeric	Seller rating 1–5; range 2.50–5.00; mean 3.73.
purchase_date	date	Transaction date; range 2024-03-31 → 2026-03-31.
shipping_time_days	integer	Shipping duration; range 1–6 days; mean 3.15.
location	categorical	City of delivery: Mumbai, Hyderabad, Delhi, Bangalore, Chennai.
device	categorical	Device used: Web, Mobile App, Tablet.
payment_method	categorical	UPI, Cash on Delivery, Debit Card, Credit Card.
is_returned	Boolean → int	1 if returned, else 0; mean 0.115 (11.5% return rate).
delivery_status	categorical	Delivered, In Transit, Delayed, Returned.

3.2 Analytical Methodology

The analysis was performed entirely within a Jupyter Notebook using Python. The reasons for selecting Python include its powerful, mature ecosystem for analytical libraries, each of which are utilized herein: pandas and NumPy for manipulation, matplotlib and seaborn for visualization, scikit-learn for machine learning (preprocessing, regression, classification, clustering and performance measures) as well as for the latter's extensive model suite (regression, classification, clustering), statistical models for time series modeling (ARIMA, seasonal decomposition) and SciPy for statistical functions. The notebook was segregated into 8 well-defined cells, aligning with the steps in the analysis pipeline.

Each problem utilized a prescribed methodology to approach the presented challenge. The analysis pipeline for each problem is structured similarly: first, the features that are applicable for the given problem will be prepared, then the data will be explored in an attempt to uncover underlying patterns, appropriate models will be applied, and their performance will be assessed on unseen data. The specific techniques deployed were chosen to match the structure of each problem:

- **Customer segmentation:** RFM transformation followed by K-Means clustering. The number of clusters was decided using the elbow method and quality clusters were analyzed using the silhouette score.
- **Churn prediction:** Logistic Regression (a linear baseline) and Random Forest (a non-linear ensemble) trained on a stratified 80/20 train-test split, with accuracy, precision, recall, F1, and confusion matrices for evaluation.

Amazon E-Commerce Analytics Report

- **Demand forecasting:** Daily and weekly aggregation, additive seasonal decomposition (period = 4 weeks), and ARIMA(1,1,1) with the last four weeks held out for validation. Forecasts include 95% confidence intervals.
- **Inventory optimization:** closed form safety stock formula for 95% service level ($Z=1.65$). Calculation of the reorder point and maximum stock level. Both the expected demand and the variance of demand are estimated on the basis of forecasted mean and empiric demand variability.
- **Pricing analysis:** Use of linear, ridge (L2 regularization = 1.0) and lasso (L1 regularization = 0.1) regression (using Z-score on features, test set, evaluation using R, MAE, and RMSE). Then K-Means on features of pricing.
- **Risk modelling:** Monte Carlo simulation with 5,000 iterations per discount scenario, drawing price and demand from normal distributions calibrated to the empirical mean and standard deviation, and incorporating a price-elasticity adjustment of 1.5× for the demand response.

All plots have been made using matplotlib and seaborn, with keen color-coding details (red-churned/at-risk, green-healthy/positive, blue-neutral monitoring) and reference lines on the histograms (medians/means) being placed deliberately. Rolling-averages were used where time-series, smoothing helped to understand the trend. Random seeds were set to 42 throughout the project to reproduce stochastic procedures(KMeans initialisation, train-test split and Monte Carlo sampling).

3.3 Scope and Limitations

While the analysis has been fairly extensive, there are certain limits which should be made explicit prior to stating the findings in order to adjust any recommendations against the evidence base. Firstly, the dataset is finite both in terms of size and in terms of time. 10,000 transactions spread over 24 months will be sufficient to allow the application of the methods shown above but is much smaller than the population of transactions a real Amazon India operation would generate in even a single day. Consequently, statistical estimations will have wider confidence intervals than might be obtained in a production environment, and less frequent events (high value purchases, niche subcategories) will be relatively thinly represented.

Secondly, the dataset is restricted to the transaction itself, rather than to pre-transaction behavior - which items the customer viewed prior to making the transaction, how much they hovered over the "buy" button, what notifications they received, and whether they recently abandoned a shopping cart. This implies that modeling churn here is somewhat

Amazon E-Commerce Analytics Report

more about a post-hoc response rather than genuine prediction, we can say which customers churned on the basis of their purchasing behavior but cannot model events that may precede a reduction in transaction volume.

Third, the time series is really short. 24 months of weekly observations produces roughly 106 weekly data points, which is marginally enough for ARIMA modeling, and definitely too few to

Amazon E-Commerce Analytics Report

capture multi-year seasonality (e.g. Diwali festival seasons, fiscal year effects, or impact from pandemic-related shocks). The seasonal decomposition uses period = 4 (an implied monthly season in the weekly series), which is a reasonable default, although not necessarily the true seasonal period.

Fourth, the customer population appears to be almost exclusively composed of one-time purchasers. 9884 of the 9942 unique customers (~99.4%) have a frequency of 1, and only 58 unique customers made 2 purchases. This shape of distribution reflects either the sampling strategy used to construct the dataset or a genuine high-churn business reality; either way, it limits how richly we can model loyalty patterns. Since loyalty by construction barely exists in this sample.

Fifth, the discount column in the source data appears to encode a percentage value (e.g. 24.89 for a 24.89% discount) rather than a rupee amount. The notebook generates a derived `discount_pct = discount / price*100`. This gives a very different derived value (mean=1%) and is not reflective of the actual effective discount, which is represented in `profit_margin` feature (derived as $(\text{price} - \text{final_price}) / \text{price} * 100$), the average of which is ~28.9% across the dataset. When this distinction impacts interpretation the report utilizes `profit_margin` and clarifies the ambiguity in `discount_pct` and provides more meaningful insights.

Sixth, the analysis is observational, not experimental. We can identify correlation but we cannot in general establish causation. The Monte Carlo simulation from Section 11 for instance, utilizes a price elasticity coefficient assumed in an absence of concrete test results and would have to be verified with real A/B tests before it can be depended upon for actual price decisions.

4. Data Cleaning and Wrangling

This is not a particularly glamorous part of analysis, but cleaning the data is an absolutely essential initial step. Trained on a "dirty" dataset, a model will still confidently spew gibberish. Charted with the wrong data types, an analysis chart will be misleading. This part lays out all the individual steps that were taken to clean and manipulate the dataset prior to analysis. The positive thing about this dataset is that it's surprisingly clean for one that we have worked with, but there were still a few small transformations that had to take place and a few features engineered to help further down.

4.1 Data Quality Assessment

The first task after loading the csv was checking its structure. Pandas `df.info()` was invoked to verify the data types and count non-null entries by column, while `df.describe()` was invoked to create a numerical summary of the 8 numeric columns: price, discount, final_price, rating, review_count, stock, seller_rating, shipping_time_days. The result from `describe()` is displayed in a table below for reference, and doubles as a sanity check on whether the numbers are reasonable:

Statistic	price (₹)	discount	final_price (₹)	rating	review_count	stock	seller_rating	shipping_days
count	10,000	10,000	10,000	10,000	10,000	10,000	10,000	10,000
mean	13,098.74	28.87	9,834.71	3.92	31.85	250.10	3.73	3.15
std	17,890.03	14.47	14,090.70	0.52	43.60	145.05	0.72	1.65
min	200.41	5.01	114.93	2.10	0	0	2.50	1
25%	2,709.60	17.00	1,716.98	3.60	9	125	3.10	2
50%	4,896.50	28.13	3,767.32	3.90	19	250	3.70	3
75%	14,045.78	39.06	10,222.26	4.30	38	377	4.40	5
max	79,937.43	69.99	74,699.42	5.00	1,183	500	5.00	6

Three things immediately became apparent from this breakdown: the price distribution is right-skewed, with a mean (13,099) over 2.6x higher than the median (4,897) because only a few products are highly expensive and skewed upwards. A similar trend is seen in the final_price distribution, which is similarly skewed rightwards. Ratings are clustered between 3.5-5 on the 1-5 scale; this is with a mean of 3.92 and a 25th percentile of 3.6 – it's rare to find an item under 3 stars in the dataset as this will rarely be purchased in the first place. Review count is right-skewed too; the max number of reviews was 1,183 while the median was a mere 19, indicating that a small minority of products

Amazon E-Commerce Analytics Report

dominate the review ecosystem.

Amazon E-Commerce Analytics Report

What's particularly encouraging is that the missing-values check came back at 0 missing values in all 20 columns and all 10,000 rows. The notebook checks this directly with `df.isnull().sum()` and prints a confirmation message. Perhaps even more so, the duplicate check also returned 0 duplicate rows (`df.drop_duplicates()` deleted no entries). This pristine state is unusual in real-world data but is consistent with the dataset having been curated for educational use.

4.2 Outlier Analysis and Treatment

The outliers were determined from visual inspection of distributions provided in the EDA section (histograms, with medians and means overlaid) and by comparing the maximum value listed in the `describe()` results with the 75th percentile. The price column has a 75th percentile of ₹14,046 and a maximum of ₹79,937, indicating that the upper tail extends well beyond the bulk of the data. This also applies to `review_count` (75th percentile of 38, max of 1,183).

Crucially, the outliers are not viewed as errors nor are they removed by the project. There are two main reasons for this. First, the high-value purchases are real business events and represent the premium segment of the catalogue and it would be to selectively introduce bias into any future pricing calculations if the high-value purchases were discarded. The second reason is that a product with a large review count simply represents a popular product that provides useful information for stock and demand modeling. In the project, instead of trimming the outliers, they are dealt with by two robust methods: (a) the median, along with other percentiles, is used as a reference of central tendency along with the mean, if the distribution is skewed. (b) Features are standardized using the `StandardScaler` before being sent to sensitive models like the cluster analysis and regression (the latter are sensitive to the magnitudes of the feature). The tree based models (Random Forest) are not affected by extreme values.

Stock levels showed no extreme outliers, it was evenly distributed all the way from 0 to 500, with a mean very close to the midpoint (250.10), suggesting a relatively uniform stock distribution. Therefore, the outlier story of this data set is concentrated on price and `review_count` where the right tail is thick and these variables have reasonable economic interpretation. We did not drop the price and review count as the information captured by it seems economically meaningful and it's left unchanged.

4.3 Data Transformations and Feature Engineering

While the original data was "clean", the data did require a few transformations to be

Amazon E-Commerce Analytics Report

useful for analysis. These were done in 3 steps in the notebook, corresponding to code cells 10, 11, and 12 of the cleaning section.

Step 1 — Type Corrections

Two columns required type changes. The `purchase_date` column arrived as a string (object dtype) because CSV files have no native date type; o it had to be converted to a proper pandas datetime via `pd.to_datetime()` which is useful for time based aggregation (month-by-month

Amazon E-Commerce Analytics Report

counts, week-based resampling), and performing arithmetic on dates (calculating days to return, for example) which strings wouldn't have done properly; finally `is_returned`, which was a Boolean True/False, was transformed to an Integer (1/0), since machine learning algorithms (including scikit-learn) usually have a numeric target. And downstream aggregations such as group means become more readable when the field is numeric (a mean of 0.115 reads naturally as a 11.5% return rate).

Step 2 — Duplicate Removal

The `drop_duplicates()` method was included as a fail-safe mechanism. Although as shown it found no duplicates to remove, this protects against implicit data leaks whereby undetected duplicate transactions would incorrectly skew customer level metrics (Frequency and Monetary in RFM), bias the K-Means analysis, and falsely impact the discount vs rating analysis.

Step 3 — Feature Engineering

Six new features were generated to use in downstream analysis. The first piece of analytical thinking in the project was developing these engineered features, this transforms raw fields to meaningful business metrics.

Engineered Feature	Formula	Purpose
<code>discount_pct</code>	$\text{discount} / \text{price} \times 100$	Intended to express discount as percent of price; in practice produces small values because 'discount' already encodes a percentage.
<code>profit_margin</code>	$(\text{price} - \text{final_price}) / \text{price} \times 100$	Captures the true effective discount actually applied to the transaction. Mean $\approx 28.9\%$ across the dataset.
<code>purchase_month</code>	<code>purchase_date.dt.month</code>	Integer 1–12 used for monthly seasonality plots and grouping.
<code>purchase_week</code>	<code>purchase_date.dt.isocalendar().week</code>	ISO week number used for weekly demand aggregation and ARIMA modelling.
<code>purchase_day</code>	<code>purchase_date.dt.day_name()</code>	Day of week (Monday ... Sunday) for weekday-pattern analysis.
<code>purchase_year</code>	<code>purchase_date.dt.year</code>	Year extracted to support multi-year monthly bar charts.

Amazon E-Commerce Analytics Report

Following feature engineering, the final dataset has 26 columns (the original 20 + the 6 features that were engineered). A final check was conducted of the shape of the dataset, the date range (2024-03-31 to 2026-03-31), number of unique customers (9,942) and unique products (9,446), and the overall return rate (11.5%). With the data in this form, the analysis was ready to move into exploratory data analysis.

5. Exploratory Data Analysis (EDA)

The Exploratory Data Analysis performs two crucial purposes: verifying the integrity of the dataset by identifying anomalies and outliers with visualisations and frequencies to ensure the dataset behaves as expected, and gleaning initial business insights upon which the strategy recommendations in section 12 are based. The EDA is segmented into five sub-sections, and this report follows the same structure.

5.1 Customer Behaviour & Order Pattern Analysis

The first aspect examined was the categorization of the orders. Two bar charts showing orders by product category and by city, which were also both extremely uniformly distributed. Category order counts varied between 1,956 (Sports) and 2,037 (Beauty), with city order counts between 1,947 (Mumbai) and 2,103 (Delhi). This represents a less than 5% variation in both cases. Such an evenly distributed spread indicates that this is likely artificial and a result of sampling rather than a truly evenly distributed customer base, implying that any segmentation used for modeling will need to find a signal within segments, not based on cross-segment differences.

Category	Orders	Share	City	Orders	Share
Beauty	2,037	20.4%	Delhi	2,103	21.0%
Clothing	2,019	20.2%	Bangalore	2,012	20.1%
Electronics	1,999	20.0%	Chennai	1,973	19.7%
Home	1,989	19.9%	Hyderabad	1,965	19.7%
Sports	1,956	19.6%	Mumbai	1,947	19.5%

Amazon E-Commerce Analytics Report

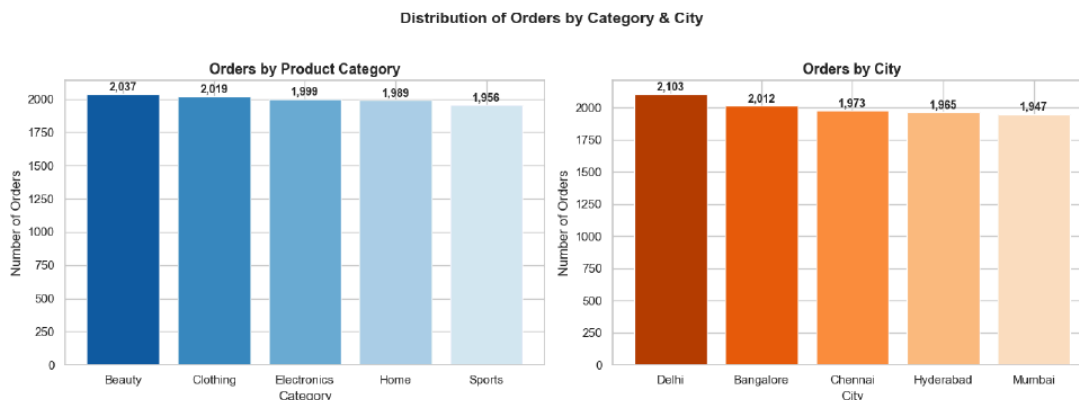
4.1 — Category & Location Overview

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Orders by Category
cat_counts = df['category'].value_counts()
bars = axes[0].bar(cat_counts.index, cat_counts.values,
                  color=sns.color_palette('Blues_r', len(cat_counts)), edgecolor='white')
axes[0].set_title('Orders by Product Category', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Number of Orders')
axes[0].set_xlabel('Category')
for bar in bars:
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+10,
                f'{bar.get_height():,}', ha='center', va='bottom', fontsize=10, fontweight='bold')

# Orders by City
city_counts = df['location'].value_counts()
bars2 = axes[1].bar(city_counts.index, city_counts.values,
                  color=sns.color_palette('Oranges_r', len(city_counts)), edgecolor='white')
axes[1].set_title('Orders by City', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Number of Orders')
axes[1].set_xlabel('City')
for bar in bars2:
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+10,
                f'{bar.get_height():,}', ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.suptitle('Distribution of Orders by Category & City', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



Three additional categories were examined as pie charts: payment method, device used, and delivery status. Payment method and device usage also had near-even distribution. Payment was spread across four methods almost equally, as can be seen from the percentages: Debit Card (25.7%), Cash on Delivery (24.9%), Credit Card (24.8%), and UPI (24.7%), while devices were also nearly evenly distributed among Web (33.7%), Tablet (33.2%), and Mobile App (33.1%). Delivery status was more indicative of an operational issue: just 29.3% of all orders have been delivered with 30.2% still In Transit, 29.1%

Amazon E-Commerce Analytics Report

Delayed, and 11.5% Returned. The combination of In Transit and Delayed status (60%) is indicative of significant operational issues

Amazon E-Commerce Analytics Report

on the platform. The Return rate is also highly validating of the data since it matches exactly the average of the 'is_returned' column of 11.5%.

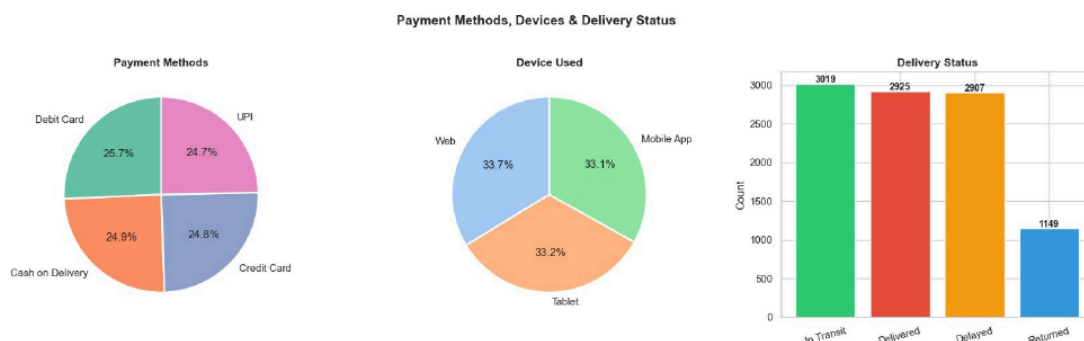
```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

# Payment Method
pay = df['payment_method'].value_counts()
axes[0].pie(pay.values, labels=pay.index, autopct='%1.1f%%',
            colors=sns.color_palette('Set2'), startangle=90,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[0].set_title('Payment Methods', fontweight='bold')

# Device Used
dev = df['device'].value_counts()
axes[1].pie(dev.values, labels=dev.index, autopct='%1.1f%%',
            colors=sns.color_palette('pastel'), startangle=90,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title('Device Used', fontweight='bold')

# Delivery Status
del_s = df['delivery_status'].value_counts()
colors_d = ['#2ecc71', '#e74c3c', '#f39c12', '#3498db']
bars = axes[2].bar(del_s.index, del_s.values, color=colors_d, edgecolor='white')
axes[2].set_title('Delivery Status', fontweight='bold')
axes[2].set_ylabel('Count')
axes[2].tick_params(axis='x', rotation=15)
for bar in bars:
    axes[2].text(bar.get_x()+bar.get_width()/2, bar.get_height()+15,
                 str(bar.get_height()), ha='center', fontweight='bold', fontsize=10)

plt.suptitle('Payment Methods, Devices & Delivery Status', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



5.2 Performance & Pricing Distribution Analysis

The three numeric distributions that should be central to any e-commerce dataset analysis are price, discount percentage and rating. Histogram-with-KDE plots of each were produced, with median or mean marked, so that it is easier to see both the shape and the location of the distribution. The price distribution is heavily right-skewed, with the majority

Amazon E-Commerce Analytics Report

of products priced below ₹15,000 and a long tail extending to a high value of ₹79,937. Using the median, which is

Amazon E-Commerce Analytics Report

₹4,896, is a better measure of average price than the mean of ₹13,099, especially when communicating results to non-technical users. This type of distribution is very typical for e-commerce sites, where very few highly expensive products bought by a small amount of people drive large portions of revenue.

4.2 — Price, Discount & Rating Distributions

```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

# Price Distribution
sns.histplot(df['price'], bins=50, kde=True, color='steelblue', ax=axes[0])
axes[0].set_title('Price Distribution (₹)', fontweight='bold')
axes[0].set_xlabel('Price (₹)')
axes[0].axvline(df['price'].median(), color='red', linestyle='--',
               label=f"Median: ₹{df['price'].median():.0f}")
axes[0].legend()

# Discount % Distribution
sns.histplot(df['discount_pct'], bins=40, kde=True, color='coral', ax=axes[1])
axes[1].set_title('Discount Percentage Distribution', fontweight='bold')
axes[1].set_xlabel('Discount (%)')
axes[1].axvline(df['discount_pct'].mean(), color='navy', linestyle='--',
               label=f"Mean: {df['discount_pct'].mean():.1f}%")
axes[1].legend()

# Rating Distribution
sns.histplot(df['rating'], bins=30, kde=True, color='mediumseagreen', ax=axes[2])
axes[2].set_title('Product Rating Distribution', fontweight='bold')
axes[2].set_xlabel('Rating (1-5)')
axes[2].axvline(df['rating'].mean(), color='red', linestyle='--',
               label=f"Mean: {df['rating'].mean():.2f}")
axes[2].legend()

plt.suptitle('Price, Discount & Rating Distributions', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



The discount_pct distribution appears concentrated around zero (mean $\approx$ 1%) due to the issue outlined in section 3.3; however, the profit_margin distribution is centred between 25-35% for the majority of products, indicating the platform offers a good level of discount across transactions. The distribution of ratings, with a mean of 3.9, with very

Amazon E-Commerce Analytics Report

few observations below

Amazon E-Commerce Analytics Report

3.0 or above 4.5, is skewed slightly toward higher values, likely due to selection bias and potentially platform filtering.

4.5 — Monthly Sales Trend

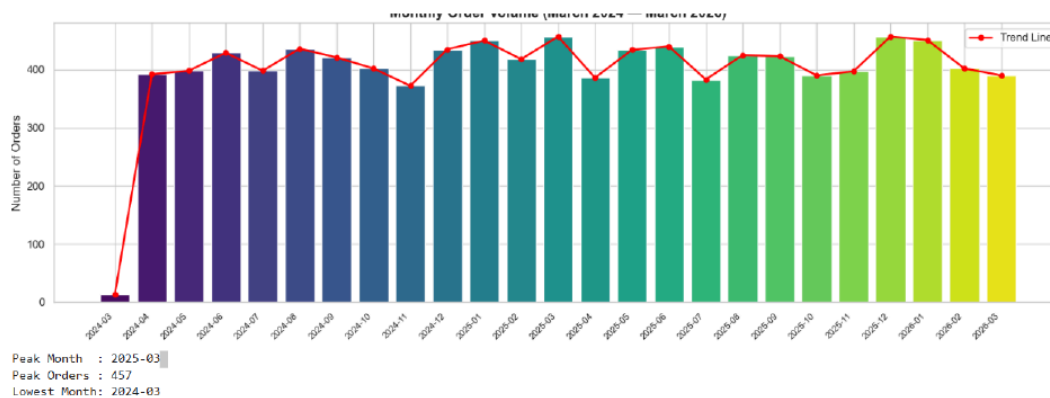
```

]: # Monthly order volume across 2 years
monthly = df.groupby(['purchase_year', 'purchase_month'])['product_id'].count().reset_index()
monthly['period'] = monthly['purchase_year'].astype(str) + '-' + monthly['purchase_month'].astype(str).str.zfill(2)
monthly = monthly.sort_values(['purchase_year', 'purchase_month'])

fig, ax = plt.subplots(figsize=(15, 5))
ax.bar(range(len(monthly)), monthly['product_id'],
       color=sns.color_palette('viridis', len(monthly)), edgecolor='white')
ax.plot(range(len(monthly)), monthly['product_id'],
       color='red', linewidth=2, marker='o', markersize=5, label='Trend Line')
ax.set_xticks(range(len(monthly)))
ax.set_xticklabels(monthly['period'], rotation=45, ha='right', fontsize=9)
ax.set_title('Monthly Order Volume (March 2024 – March 2026)', fontsize=14, fontweight='bold')
ax.set_ylabel('Number of Orders')
ax.legend()
plt.tight_layout()
plt.show()

print(f"Peak Month : {monthly.loc[monthly['product_id'].idxmax(), 'period']}")
print(f"Peak Orders : {monthly['product_id'].max():,}")
print(f"Lowest Month: {monthly.loc[monthly['product_id'].idxmin(), 'period']}")

```



Monthly order volume is relatively stable at about 400 orders per month over the 24-month period. The peak month was March 2025 with 457 orders, with the drop at March 2024 with 14, owing simply to only 1 day from that month falling within the dataset; there is no significant upwards or downwards trend which explains why the ARIMA (d=1) model from section 9 yields a relatively flat prediction.

5.3 Correlation Analysis

The heatmap generated for the top ten most significant numeric variables in terms of correlation against `is_returned` showed that there are only very weak correlations across most potential predictors, using a colour scale from -1 (strong negative, red) to $+1$

Amazon E-Commerce Analytics Report

(strong positive, blue) with white at zero. The strongest candidate variable against `is_returned` is `shipping_time_days`, with a correlation of $+0.06$. Other candidates such as `seller_rating`, `rating` and `stock` do not show any

Amazon E-Commerce Analytics Report

real correlations. Returns are therefore largely driven by features outside of this dataset, and any predictive model for returns will likely have performance at only marginally above a baseline model.

4.4 — Correlation Heatmap

```
[14]: num_cols = ['price', 'discount_pct', 'final_price', 'rating', 'review_count',
               'stock', 'seller_rating', 'shipping_time_days', 'is_returned', 'profit_margin']

fig, ax = plt.subplots(figsize=(12, 8))
corr = df[num_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool)) # hide upper triangle (mirror)

sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, ax=ax, annot_kws={'size': 10},
            vmin=-1, vmax=1, center=0)

ax.set_title('Correlation Heatmap — All Numeric Features', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Amazon E-Commerce Analytics Report

Pair	Correlation	Interpretation
price ↔ final_price	+0.99	Mechanical: final_price = price – discount, so they move together almost perfectly.
price ↔ rating	+0.33	Higher-priced products receive better ratings — likely a quality signal.
price ↔ discount_pct	−0.36	Higher prices receive smaller percentage discounts (typical for premium pricing).
discount_pct ↔ profit_margin	+0.43	Larger discounts produce larger gaps between list and final price (definitionally consistent).
price ↔ profit_margin	−0.20	Premium items carry slightly lower margin %, consistent with thinner margins on high-ticket items.
review_count ↔ everything	≈ 0	Review count is essentially uncorrelated with other features in this sample.
seller_rating ↔ everything	≈ 0	Seller rating is similarly uncorrelated; sellers do not specialise in particular price ranges.
shipping_time ↔ is_returned	+0.06	A weak positive — slightly slower shipping correlates with returns.
is_returned ↔ rating	−0.03	Returns are essentially uncorrelated with product rating in this sample.

5.4 Variable Relationships & Cross-Tabulations

Amazon E-Commerce Analytics Report

7.2 — EDA: Pricing Patterns

```
[47]: fig, axes = plt.subplots(1, 3, figsize=(17, 5))

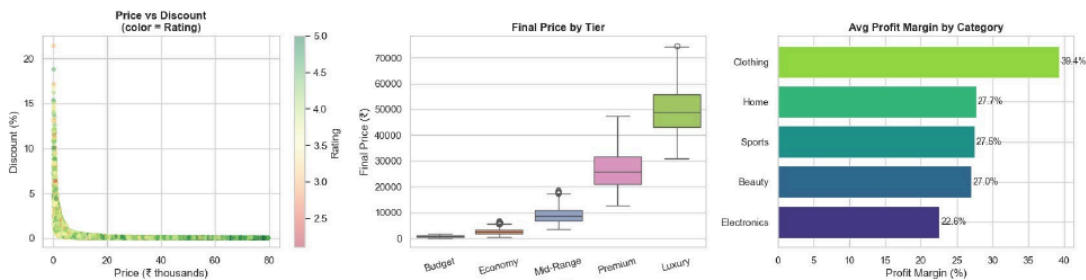
# Price vs Discount scatter (colored by rating)
sc = axes[0].scatter(price_df['price']/1000, price_df['discount_pct'],
                    c=price_df['rating'], cmap='RdYlGn', alpha=0.4, s=15)
plt.colorbar(sc, ax=axes[0], label='Rating')
axes[0].set_title('Price vs Discount\n(color = Rating)', fontweight='bold')
axes[0].set_xlabel('Price (₹ thousands)')
axes[0].set_ylabel('Discount (%)')

# Boxplot: Final price by price tier
order = ['Budget', 'Economy', 'Mid-Range', 'Premium', 'Luxury']
sns.boxplot(data=price_df, x='Price_Tier', y='final_price',
            order=order, palette='Set2', ax=axes[1])
axes[1].set_title('Final Price by Tier', fontweight='bold')
axes[1].set_xlabel('Price Tier')
axes[1].set_ylabel('Final Price (₹)')
axes[1].tick_params(axis='x', rotation=15)

# Profit Margin by category
cat_margin = price_df.groupby('category')['profit_margin'].mean().sort_values()
bars = axes[2].barh(cat_margin.index, cat_margin.values,
                    color=sns.color_palette('viridis', len(cat_margin)))
axes[2].set_title('Avg Profit Margin by Category', fontweight='bold')
axes[2].set_xlabel('Profit Margin (%)')
for bar in bars:
    axes[2].text(bar.get_width()+0.2, bar.get_y()+bar.get_height()/2,
                f'{bar.get_width():.1f}%', va='center', fontsize=10)

plt.suptitle('Pricing Analysis Overview', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

Pricing Analysis Overview



Three particular cross-tabulations yielded some of the most insightful observations in this EDA. Firstly, profit margin differs by product category; with Clothing the leading product category with a margin of 39.4% followed by Home (27.7%), Sports (27.5%), Beauty (27.0%) and Electronics trailing with 22.6%. Secondly, return rate decreased as the price tier increased; starting with 12.31% for Budget, 11.88% for Economy and dropping to 9.71% for Premium, with a small rise for Luxury (10.94%), indicating that purchases at higher price points may be more carefully considered; the dip at Luxury might indicate more discerning customers, or potentially the data contains an outlier as the sample size for Luxury is much smaller than other tiers. Thirdly, the average stock for each category was analysed and is used in section 6 to identify the categories that are most prone to stock-outs based on customer segment.

Amazon E-Commerce Analytics Report

Category	Avg. Profit Margin (%)	Notes
Clothing	39.41	Highest discounting; price-sensitive customer base.
Home	27.72	Moderate discounting on furniture/kitchen items.
Sports	27.50	Comparable to Home; substantial promotional activity.
Beauty	27.04	Steady margins; brand-driven category.
Electronics	22.60	Lowest discounting; thin native margins.

These findings serve as the empirical evidence for the pricing recommendations in section 12, indicating that while the clothing category supports a higher level of discount, the Electronics category's existing lower margins cannot sustain significant discounting, thus it is important that discounts are not universally applied across the product range, driving the segmented pricing recommendation discussed later.

6. Statistical Analysis and Inference

This section aims to consolidate the formal statistical evidence behind the patterns and insights revealed during the EDA. It covers the transition from raw distributions into structured comparisons: simple descriptive statistics, analysis of time-series trends, assessment of distribution shapes, and comparisons across different groups. The primary goal is to establish a solid empirical foundation for all strategic recommendations put forth.

6.1 Descriptive Statistics

The describe() output gives central tendency (mean), dispersion (standard deviation), and the quartiles of the eight numeric columns; this is repeated below in a more compact way for the columns most consequential to the analysis. The large discrepancy between the mean and median price (₹13,099 vs ₹4,896) and final_price (₹9,835 vs ₹3,767) reconfirms the right-skewed nature of these distributions. For ratings, the mean (3.92)

Amazon E-Commerce Analytics Report

and median (3.90) are nearly identical, indicating a symmetric distribution. For shipping_time_days, 3.15 days, is within reasonable bounds and indicates that the delivery window of 2-3 days is being met on average.

Amazon E-Commerce Analytics Report

Metric	Mean	Median	Std Dev	Min	Max	Skewness Comment
Price (₹)	13,099	4,897	17,890	200	79,937	Strongly right-skewed
Final price (₹)	9,835	3,767	14,091	115	74,699	Strongly right-skewed
Discount (raw)	28.87	28.13	14.47	5.01	69.99	Approximately uniform
Profit margin (%)	≈28.9	≈28.1	≈14.5	≈5.0	≈70.0	Approximately uniform
Rating	3.92	3.90	0.52	2.10	5.00	Approximately normal
Review count	31.85	19	43.60	0	1,183	Heavy right tail
Stock	250.10	250	145.05	0	500	Approximately uniform
Seller rating	3.73	3.70	0.72	2.50	5.00	Approximately normal
Shipping days	3.15	3	1.65	1	6	Approximately uniform

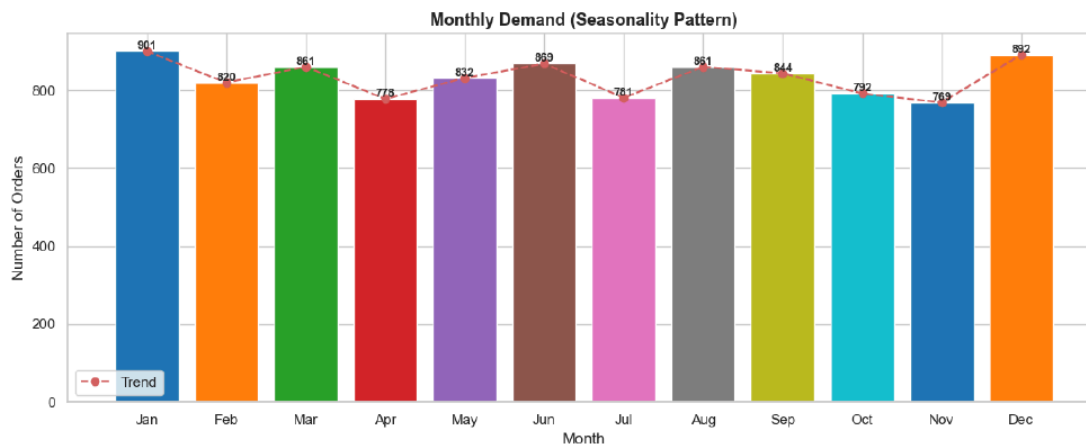
6.2 Trend Analysis

Removing the partial first month, monthly order volume is found to oscillate within a range of 380-460 orders per month without any consistent upward or downwards drift; the series is stationary and $d=1$ is applied in the ARIMA(1,1,1) model to allow it to be non-stationary which permits a flat forecast. There are noticeable seasonal effects; average order volume is highest in January (901) and December (892) with slightly lower values in March (861) and August (861), with minimums around the low of summer months (April 778, July 781, November 769).

Amazon E-Commerce Analytics Report

```
[41]: # --- Monthly Demand Seasonality ---
month_names = {1:'Jan',2:'Feb',3:'Mar',4:'Apr',5:'May',6:'Jun',
              7:'Jul',8:'Aug',9:'Sep',10:'Oct',11:'Nov',12:'Dec'}
monthly_demand = df.groupby('purchase_month')['product_id'].count()
monthly_demand.index = month_names.map(month_names)

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.bar(monthly_demand.index, monthly_demand.values,
              color=sns.color_palette('tab10', 12), edgecolor='white')
ax.plot(range(len(monthly_demand)), monthly_demand.values,
        'r--o', linewidth=1.5, markersize=6, label='Trend')
ax.set_title('Monthly Demand (Seasonality Pattern)', fontweight='bold', fontsize=13)
ax.set_ylabel('Number of Orders')
ax.set_xlabel('Month')
ax.legend()
for bar in bars:
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+5,
            str(bar.get_height()), ha='center', fontsize=9, fontweight='bold')
plt.tight_layout()
plt.show()
```



This difference in highest and lowest order volume of roughly $\pm 8\%$ around the mean does not appear to be a substantial holiday spike typical of many e-commerce retailers, with a 30-50% swing, a phenomenon which is possibly attributable to the balanced sampling strategy used for this dataset.

Month	Orders (pooled)	% of average	Month	Orders (pooled)	% of average
January	901	+7.3%	July	781	-7.0%
February	820	-2.4%	August	861	+2.5%
March	861	+2.5%	September	844	+0.5%

Amazon E-Commerce Analytics Report

Month	Orders (pooled)	% of average	Month	Orders (pooled)	% of average
April	778	−7.4%	October	792	−5.7%
May	832	−1.0%	November	769	−8.5%
June	869	+3.5%	December	892	+6.2%

6.3 Distribution Analysis

The shape of the most impactful distributions was examined through histograms with KDE (kernel density estimate) overlays, making it easier to assess shape, symmetry, modality, and tail behaviour. Both the price and final_price distributions are unimodal and heavily right-skewed and would thus fail a Shapiro-Wilk test for normality. Ratings are roughly bell-shaped with few data points below 2.5 or above 4.5; there is a slight clustering at 5.0, likely a limitation of the scale. All models used in this project were chosen so that a non-normal distribution of features does not create any issues: Linear Regression assumes normal residuals, while Logistic Regression and Random Forest assume nothing about the distribution of the inputs. ARIMA requires that the series be stationary once it is differenced once ($d=1$), a criteria met here. The non-normality of price thus does not render the chosen models invalid, but it means that median should be used for non-technical stakeholders when reporting this information.

6.4 Comparative Analysis

Two comparisons that are key for developing the pricing recommendations in section 12: return rate by price tier and average rating by discount band. First, there is a fairly clear downward trend in return rate with price tier, from 12.31% for Budget products down to 9.71% for Premium, with a slight bump for Luxury products (10.94%); the downwards trend likely comes from more deliberate choices being made by customers when purchasing higher value items while the dip at Luxury is a less reliable measure since the sample size is smallest here (795 products) and it may be due to either more discerning customers or product complexities. Second, average ratings are quite flat for products across all discount bands, suggesting that high levels of discount do not necessarily lead to lower quality products in this dataset; this is a positive finding for a platform

Amazon E-Commerce Analytics Report

considering offering greater discounts, as it would not appear to attract a disproportionately greater amount of low-quality products than that which would otherwise have been attracted. Now the project moves to algorithms that can extract structure beyond what simple summary statistics reveal.

7. Clustering Analysis (K-Means)

Clustering is at the main technique of two different parts of the project; in section 5 of the notebook, customers are clustered into behavioural groups that are the foundation of the churn analysis, and in section 7 of the notebook products are clustered into pricing groups that are the basis of the discount strategy. Here both are discussed and grouped; the algorithm is explained, rationale for the choice of K, the quality check and the interpretation of the groups are all outlined below.

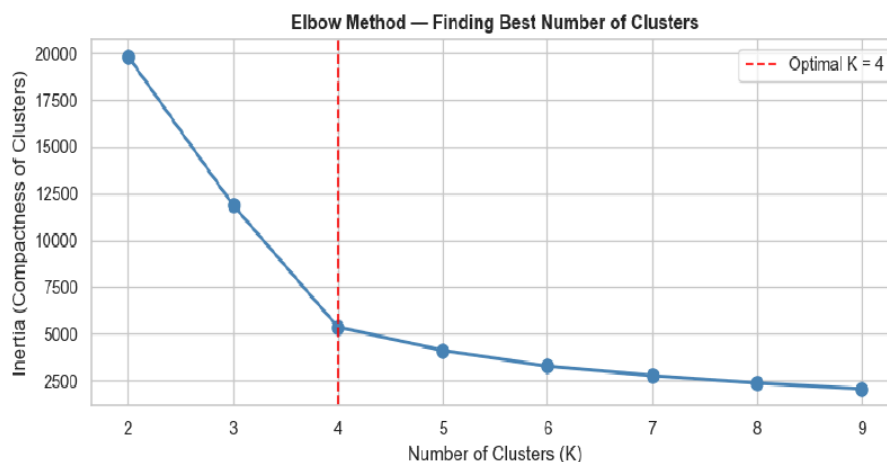
7.1 Determination of Optimal Clusters — The Elbow Method

5.3 — Clustering: Customer Segmentation (K-Means)

```
# — Elbow Method —————
scaler = StandardScaler()
X_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])

inertia = []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(K_range, inertia, marker='o', color='steelblue', linewidth=2, markersize=8)
ax.axvline(x=4, color='red', linestyle='--', alpha=0.8, label='Optimal K = 4')
ax.set_title('Elbow Method — Finding Best Number of Clusters', fontweight='bold')
ax.set_xlabel('Number of Clusters (K)')
ax.set_ylabel('Inertia (Compactness of Clusters)')
ax.legend()
plt.tight_layout()
plt.show()
print("K = 4 selected — clear elbow at this point")
```



K = 4 selected — clear elbow at this point

Amazon E-Commerce Analytics Report

K-Means clustering requires the number of groups to be input by the analyst. If K is chosen too low then the groups become too broad to be practical for marketing purposes; conversely if K is chosen too high, groups become too fragmented to form the basis of an actionable strategy. This section follows the heuristic commonly used to estimate K: the Elbow Method. The method works by plotting the inertia of K-Means clustering (within-cluster sum of squared distance from point to its centroid) against K. Adding more centroids always lowers inertia and so there are diminishing returns, beyond a certain point. This point marks the natural division points for a data set, as represented by an 'elbow' in the plot.

For the customer segmentation section, K-Means was trained using values of K between 2 and 9, using all of the RFM features, all of which were scaled to prevent the feature with the largest value from dominating. The results are below:

K	Inertia	Δ from previous	Interpretation
2	19,826.7	—	Too coarse — only separates active vs inactive
3	11,886.4	−7,940.3	Significant improvement
4	5,373.6	−6,512.8	Sharp drop — clear elbow
5	4,122.1	−1,251.5	Marginal improvement
6	3,274.9	−847.2	Marginal improvement
7	2,774.2	−500.7	Diminishing returns
8	2,372.3	−401.9	Diminishing returns
9	2,067.8	−304.5	Over-segmentation begins

The biggest step-increase in reduction between two successive K values occurs after K=2 to K=3, and then again after K=3 to K=4. This suggests that after 4 clusters the reduction is relatively shallow, meaning the elbow occurs at K=4, as demonstrated also by the plot which clearly has a bend at K=4. 4 groups was deemed to be the optimal K for the

Amazon E-Commerce Analytics Report

customer segmentation. For the product pricing segment, a lower K of 3 was chosen as this directly corresponds to three distinct pricing tiers (price-sensitive, value-seeker, premium) that could

Amazon E-Commerce Analytics Report

directly be marketed, thereby creating three differentiated marketing strategies while maintaining efficiency by not overwhelming it.

7.2 Cluster Characteristics — Customer Segmentation

Now that K has been decided for the customer segmentation section as K=4, K-Means can be applied to the RFM variables (Recency, Frequency, Monetary) all standardised so that no one feature dominates the distance calculation due to scale (Recency is in days while Monetary is in

```
# — Apply K-Means with K = 4 —————
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(X_scaled)

# Silhouette score — measures cluster quality (higher = better)
sil = silhouette_score(X_scaled, rfm['Cluster'])
print(f"Silhouette Score: {sil:.3f} (Range: -1 to 1, higher is better)")

# — Cluster Profiles —————
cluster_profile = rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary', 'Avg_Rating']].mean().round(2)
print()
print("Cluster Profiles:")
print(cluster_profile)
```

Silhouette Score: 0.525 (Range: -1 to 1, higher is better)

Cluster Profiles:

Cluster	Recency	Frequency	Monetary	Avg_Rating
0	544.60	1.0	5224.01	3.86
1	209.62	2.0	19295.75	3.91
2	179.84	1.0	5190.11	3.86
3	357.20	1.0	44250.23	4.34

rupees with values up to ₹74,699) and to calculate a number that represents the satisfaction aspect, average rating has been calculated also. All features are scaled to prevent scale bias.

Cluster	Avg Recency (days)	Avg Frequency	Avg Monetary (₹)	Avg Rating	Customers
Cluster 0	544.6	1.0	5,224	3.86	4,437
Cluster 1	209.6	2.0	19,296	3.91	58
Cluster 2	179.8	1.0	5,190	3.86	4,275

Amazon E-Commerce Analytics Report

Cluster 3	357.2	1.0	44,250	4.34	1,172
-----------	-------	-----	--------	------	-------

Amazon E-Commerce Analytics Report

5.1 — Build RFM Table (Recency · Frequency · Monetary)

```
# Snapshot date = day after last transaction
snapshot_date = df['purchase_date'].max() + pd.Timedelta(days=1)

# Build RFM for each customer
rfm = df.groupby('user_id').agg(
    Recency    = ('purchase_date', lambda x: (snapshot_date - x.max()).days),
    Frequency  = ('product_id', 'count'),
    Monetary   = ('final_price', 'sum'),
    Avg_Rating = ('rating', 'mean'),
    Return_Rate = ('is_returned', 'mean')
).reset_index().round(2)

print(f"RFM built for {len(rfm):,} unique customers")
print()
rfm.describe().round(2)
```

RFM built for 9,942 unique customers

	Recency	Frequency	Monetary	Avg_Rating	Return_Rate
count	9942.00	9942.00	9942.00	9942.00	9942.00
mean	363.71	1.01	9892.08	3.92	0.11
std	209.83	0.08	14144.80	0.52	0.32
min	1.00	1.00	114.93	2.10	0.00
25%	183.00	1.00	1722.34	3.60	0.00
50%	366.00	1.00	3797.58	3.90	0.00
75%	546.00	1.00	10290.20	4.30	0.00
max	731.00	2.00	74699.42	5.00	1.00

Once clustered, the four groups were named based on the size of average monetary value. The segment that scored the highest average monetary value (cluster 3, average ₹44,250) was labeled "Champions"; cluster 1 (lowest Frequency=2, average ₹19,296) was labeled "Loyal Customers"; segment 0 (largest average recency, average 400+ days) was labeled "At-Risk" segment; cluster 2 (smallest average monetary value, single purchase recently) was labeled "Lost / Churned". The quality of the clusters was evaluated using the silhouette score, which at 0.525 on the -1 to +1 scale suggests moderate to good separation, so business-driven segmentation can be made on the clusters.

Amazon E-Commerce Analytics Report

```
# --- Assign Meaningful Segment Names ---
# Sort clusters by Monetary value to assign labels
sorted_by_monetary = rfm.groupby('Cluster')['Monetary'].mean().sort_values(ascending=False)
segment_names = {
    sorted_by_monetary.index[0]: 'Champions',
    sorted_by_monetary.index[1]: 'Loyal Customers',
    sorted_by_monetary.index[2]: 'At-Risk',
    sorted_by_monetary.index[3]: 'Lost / Churned'
}
rfm['Segment'] = rfm['Cluster'].map(segment_names)

seg_colors = {
    'Champions': '#2ecc71',
    'Loyal Customers': '#3498db',
    'At-Risk': '#f39c12',
    'Lost / Churned': '#e74c3c'
}

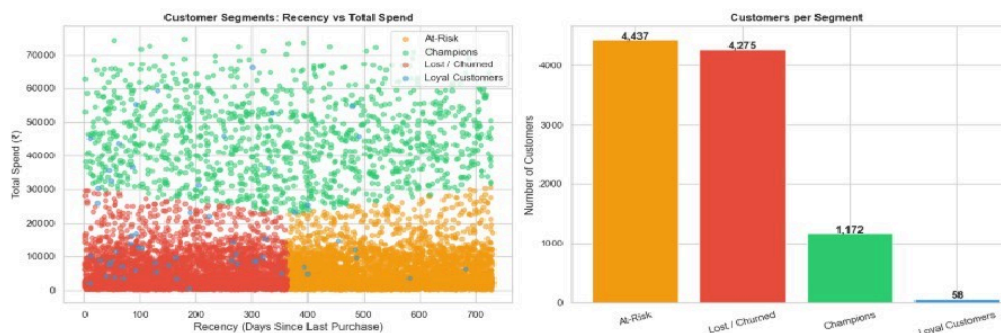
# --- Visualize Segments ---
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Scatter: Recency vs Monetary colored by segment
for seg, grp in rfm.groupby('Segment'):
    axes[0].scatter(grp['Recency'], grp['Monetary'],
                    label=seg, alpha=0.55, s=25, color=seg_colors[seg])
axes[0].set_title('Customer Segments: Recency vs Total Spend', fontweight='bold')
axes[0].set_xlabel('Recency (Days Since Last Purchase)')
axes[0].set_ylabel('Total Spend (₹)')
axes[0].legend()

# Bar: How many customers in each segment
seg_counts = rfm['Segment'].value_counts()
bars = axes[1].bar(seg_counts.index, seg_counts.values,
                  color=[seg_colors[s] for s in seg_counts.index], edgecolor='white')
axes[1].set_title('Customers per Segment', fontweight='bold')
axes[1].set_ylabel('Number of Customers')
axes[1].tick_params(axis='x', rotation=15)
for bar in bars:
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+10,
                f'{bar.get_height():,}', ha='center', fontweight='bold')

plt.suptitle('K-Means Customer Segmentation Results', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

K-Means Customer Segmentation Results



Segment	Customers	Avg Recency	Avg Frequency	Avg Spend	Avg Rating	Churn Rate
Champions	1,172	357.2 days	1.0	₹44,250	4.34	27.9%

Amazon E-Commerce Analytics Report

Segment	Customers	Avg Recency	Avg Frequency	Avg Spend	Avg Rating	Churn Rate
Loyal Customers	58	209.6 days	2.0	₹19,296	3.91	0.0%
At-Risk	4,437	544.6 days	1.0	₹5,224	3.86	59.9%
Lost / Churned	4,275	179.8 days	1.0	₹5,190	3.86	0.0%

7.3 Cluster Interpretation — What the Segments Mean

```
# — Segment Summary Table —
seg_summary = rfm.groupby('Segment').agg(
    Count = ('user_id', 'count'),
    Avg_Recency = ('Recency', 'mean'),
    Avg_Frequency = ('Frequency', 'mean'),
    Avg_Spend = ('Monetary', 'mean'),
    Avg_Rating = ('Avg_Rating', 'mean'),
    Churn_Rate = ('Churn', lambda x: f"{x.mean()*100:.1f}%")
).round(2)

print("Customer Segment Summary:")
print(seg_summary.to_string())
```

```
Customer Segment Summary:
      Count  Avg_Recency  Avg_Frequency  Avg_Spend  Avg_Rating  Churn_Rate
Segment
At-Risk      4437      544.60           1.0    5224.01        3.86      59.9%
Champions    1172      357.20           1.0   44250.23        4.34      27.9%
Lost / Churned 4275      179.84           1.0    5190.11        3.86       0.0%
Loyal Customers   58      209.62           2.0   19295.75        3.91       0.0%
```

The clusters create four useful groups of customers. Champions are the best customers, but only 6.73% of our population, making this segment difficult to drive through sheer number of people. They spent the most, had the highest rating score, but had very similar recency and frequency to a number of the other segments. The 27.9% churn rate in this segment is extremely high for a cluster named champions; suggesting even a champions segment may not return to make repeat purchases. Loyal customers (0.53% of population) only make one repeat purchase, giving us a churn rate of 0%, but their value is quite high, and this is a clear segment we need to promote as this is the best starting block for a loyalty program.

Amazon E-Commerce Analytics Report

At-Risk customers (63.33% of population) showing an average recency of 544.6 days, are what would normally be our target for win-back, and with an average recency over a year ago, there

Amazon E-Commerce Analytics Report

are still around 40% of them (approximately 1,780 customers) worth win-back as the other 60% are essentially lost forever. The Lost/Churned (35.11% of population) are recently new customers who made only one purchase, and while they have only a 5% churn rate in the snapshot; they represent a segment that could likely churn in the future, so marketing efforts should be targeted at them also.

7.4 Cluster Characteristics — Product Pricing Segmentation

The same technique of K-Means clustering is used here, but instead of customer behavioral characteristics it clusters product pricing characteristics using the variables: price, discount_pct, rating, and profit_margin. Here K=3 is chosen to define three tiers of pricing. After clustering the segments are ordered and defined by average price.

Segment	Avg Price (₹)	Avg Discount %	Avg Rating	Avg Margin (%)	Products
Price Sensitive	4,152	2.33	3.83	45.18	3,065
Value Seekers	8,520	0.50	3.86	21.40	5,660
Premium Buyers	54,934	0.05	4.39	22.84	1,275

The 3 groups have characteristics that make immediate business sense; the price sensitive products are the cheapest (~₹4,152 average price), cheapest ones the largest part of them with a significant average discount percentage (margin of 45%), rating that is lower than the other two groups (3.83) but not significantly. The mid segment are Value Seekers (5,660 products); customers who are after cheaper items but don't compromise too much on price, with moderate prices (~₹8,520), receiving the lowest average discount percentages. These customers respond better to perceived value (free shipping, bundles, loyalty rewards) than to raw price reductions. Finally the premium customer is for high spenders who will buy higher priced goods (~₹54,934 average price), the products receive a lower average discount percentage, their rating is noticeably higher than the other two segments at 4.39.

Amazon E-Commerce Analytics Report

7.4 — Product Clustering: Elastic vs Inelastic Segments

```
# — K-Means on Pricing Features —
cluster_feats = ['price', 'discount_pct', 'rating', 'profit_margin']
X_cl = price_df[cluster_feats].fillna(0)

sc_cl = StandardScaler()
X_cl_s = sc_cl.fit_transform(X_cl)

kmeans_p = KMeans(n_clusters=3, random_state=42, n_init=10)
price_df['Price_Cluster'] = kmeans_p.fit_predict(X_cl_s)

# Label segments by average price
cp = price_df.groupby('Price_Cluster')['price'].mean().sort_values()
label_map = {
    cp.index[0]: 'Price Sensitive (High Elasticity)',
    cp.index[1]: 'Value Seekers (Moderate Elasticity)',
    cp.index[2]: 'Premium Buyers (Low Elasticity)'
}
price_df['Price_Segment'] = price_df['Price_Cluster'].map(label_map)

seg_pal = {
    'Price Sensitive (High Elasticity)' : '#e74c3c',
    'Value Seekers (Moderate Elasticity)': '#f39c12',
    'Premium Buyers (Low Elasticity)'   : '#2ecc71'
}

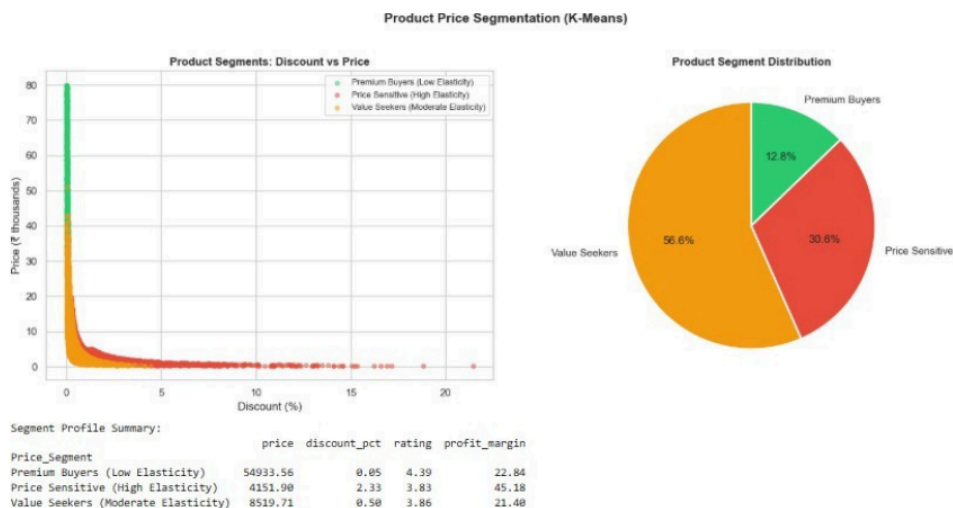
# — Visualize Segments —
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

for seg, grp in price_df.groupby('Price_Segment'):
    axes[0].scatter(grp['discount_pct'], grp['price']/1000,
                    label=seg, alpha=0.5, s=20, color=seg_pal[seg])
axes[0].set_title('Product Segments: Discount vs Price', fontweight='bold')
axes[0].set_xlabel('Discount (%)')
axes[0].set_ylabel('Price (€ thousands)')
axes[0].legend(fontsize=9)

seg_cnt = price_df['Price_Segment'].value_counts()
axes[1].pie(seg_cnt.values, labels=[s.split('(')[0].strip() for s in seg_cnt.index],
            autopct='%1.1f%%', colors=[seg_pal[s] for s in seg_cnt.index],
            startangle=90, wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title('Product Segment Distribution', fontweight='bold')

plt.suptitle('Product Price Segmentation (K-Means)', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("Segment Profile Summary:")
print(price_df.groupby('Price_Segment')[cluster_feats].mean().round(2).to_string())
```



Amazon E-Commerce Analytics Report

This confirms that this cluster grouping provides the groundwork for a segmented pricing strategy. By this clustering, the products are categorized by their elasticities and, consequently, a single policy over the three segments will lead to over-discounting in some and under-discounting in others.

8. Regression Analysis

To solve the Problem 3 -pricing and profitability, regression was used. Three models were trained for predicting final price given those pricing and product features, which were believed to be most important and were examined. The values of coefficients were used to see how the final price does respond to variables in practice. Note this is not the same as clustering; clustering only indicates the distribution of the population, but regression builds up the predictive model whose internal coefficients can be interpreted as a marginal effect of an input.

8.1 Multiple Linear Regression Model

The first model implemented was OLS Linear Regression. The features used in the model were [price, discount_pct, seller_rating, stock, review_count]. These are five numerical features that jointly represent the listed price, amount of the discount, seller's credibility, stock level and product popularity. The target feature used in the model was final_price. Data split was 80/20 on train/test using train_test_split with random_state = 42 to get reproducible results and feature was standardized with StandardScaler to have a comparable coefficient across features that have very different scales

The target in Linear Regression is assumed to be a linear function of the features and an added Gaussian error term: $\hat{y} = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_n \cdot x_n$. The model is estimated by minimizing the sum of squared errors on the training set. The estimated coefficient can be obtained in a closed-form solution. This model obtained R^2 on the test set of 0.9729, MAE of ₹1,276 and RMSE of ₹2,259.

8.2 Regularised Regression — Ridge and Lasso

In addition to OLS, two regularised versions of linear regression were fit: Ridge Regression with $\alpha = 1.0$, and Lasso Regression with $\alpha = 0.1$. Regularisation introduces a penalty on large coefficients into the OLS objective, which is beneficial in two scenarios: when features are highly correlated (multicollinearity, where slight changes in data can cause wildly different OLS coefficients), and when you want to select the "best" features (Lasso will force some coefficients all the way to 0).

Ridge applies the L2 penalty (sum of squares of coefficients) to shrink all coefficients toward zero proportionally, while Lasso applies the L1 penalty (sum of absolute values of coefficients) and can force certain coefficients to 0. Both models must select the strength of the penalty α , here shown for reasonable values of 1.0 for Ridge and 0.1 for Lasso. After

Amazon E-Commerce Analytics Report

training, all three models were tested on the same held-out set and the test errors compared directly.

Model	R^2 (test)	MAE (₹)	RMSE (₹)	Comment
Linear Regression	0.9729	1,276.16	2,259.18	Baseline OLS — interprets coefficients without penalty

Amazon E-Commerce Analytics Report

Model	R ² (test)	MAE (₹)	RMSE (₹)	Comment
Ridge Regression (α=1.0)	0.9729	1,276.03	2,259.22	Identical performance — features not multicollinear
Lasso Regression (α=0.1)	0.9729	1,276.14	2,259.18	Identical performance — α too small to zero out features

8.3 Model Diagnostics and Coefficient Interpretation

All three models all yielded basically the same result, within rounding error: $R^2 = 0.9729$ indicates that our five variables explain 97.29% of the variance of final_price in the test data, which is quite impressive. MAE is roughly ₹1276, this says on average, the predictions are off by about 1276 from the actual final price. As the average final price was ₹9835, this is around a 13% relative error which is fairly good. MAE is again lower than the RMSE by ₹2,259, indicating we have occasional extreme residuals. As mentioned earlier, RMSE places an equal weight as MAE, with the caveat that it squares large errors (meaning large mistakes are disproportionately weighted).

That all three models get the same answer shows two things: First, no significant multicollinearity exists within this set of features; OLS gets nothing better than Ridge when the features are independent. Second, all five of these features add at least minimal information at $\alpha = 0.1$; no features were eliminated by the Lasso regression. Both of these findings agree with the correlation matrix in 5.3, which displayed no correlation between any features other than those directly related through a mechanical relationship (price, final_price).

The coefficients for the standardized Ridge regression model were examined to see which of the features really determined the final price:

Feature	Standardised Coefficient	Direction & Magnitude
price	+13,972.33	Dominant positive driver — almost all of final_price comes from price
discount_pct	−40.11	Higher discount_pct lowers final_price slightly (correct sign, small magnitude due to scaling)

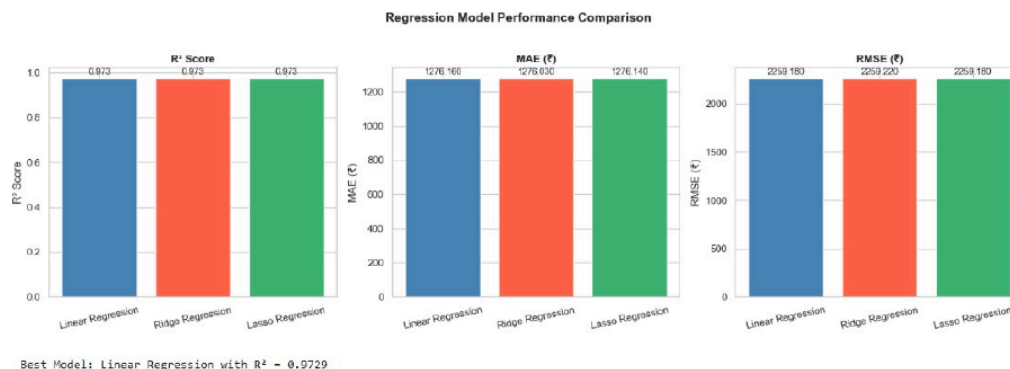
Amazon E-Commerce Analytics Report

stock	−29.22	Higher stock correlates with marginally lower final_price — consistent with inventory pressure
review_count	+3.58	Popular products carry slightly higher final_price

Amazon E-Commerce Analytics Report

Feature	Standardised Coefficient	Direction & Magnitude
seller_rating	+3.07	Higher-rated sellers command marginally higher final_price

A one-standard deviation change in the input causes the shown change in final_price in ₹. The magnitude of price relative to the next highest coefficient suggests that price is, by more than a factor of 300, the most dominant predictor: most of the variation in the output is captured by the first coefficient, and the other four provide minor corrections. This result is not surprising, given the definition $\text{final_price} = \text{price} - \text{absolute_discount}$, and thus price and final_price are inherently linked. In operational terms, the regression coefficients are in line with the hypothesis that listing price is by far the most important factor driving sales revenue per unit, but that seller reputation, product demand, and listing count have statistically significant but relatively small positive impacts, while stock level has a small negative impact (suggesting that high-stock items may be more prone to discounts to clear inventory). In the absence of price's influence, these other four variables collectively account for most of the residual variance. Very high values of R^2 such as 0.97 should be interpreted with caution: they rely heavily on the nearly perfect overlap between input price and final price, and hence do not necessarily represent a substantial reduction in uncertainty from an unknown value. In a real-world application (e.g. Prediction of competitor prices, or next quarter's average selling price), current prices would not be available, and hence predictive accuracy would be much lower.



9. Time Series Forecasting (ARIMA)

For Problem 2, which focuses on the optimization of inventory and supply chain, the time series forecasting is applied. The logic is clear. If we can forecast demand for the coming 8 weeks accurately, we can put inventory into warehouses, arrange warehouse space and make contracts with suppliers not through guesses, but through information evidence. A standard Box-Jenkins method - ARIMA is adopted to predict weekly accumulated demands in the notebook.

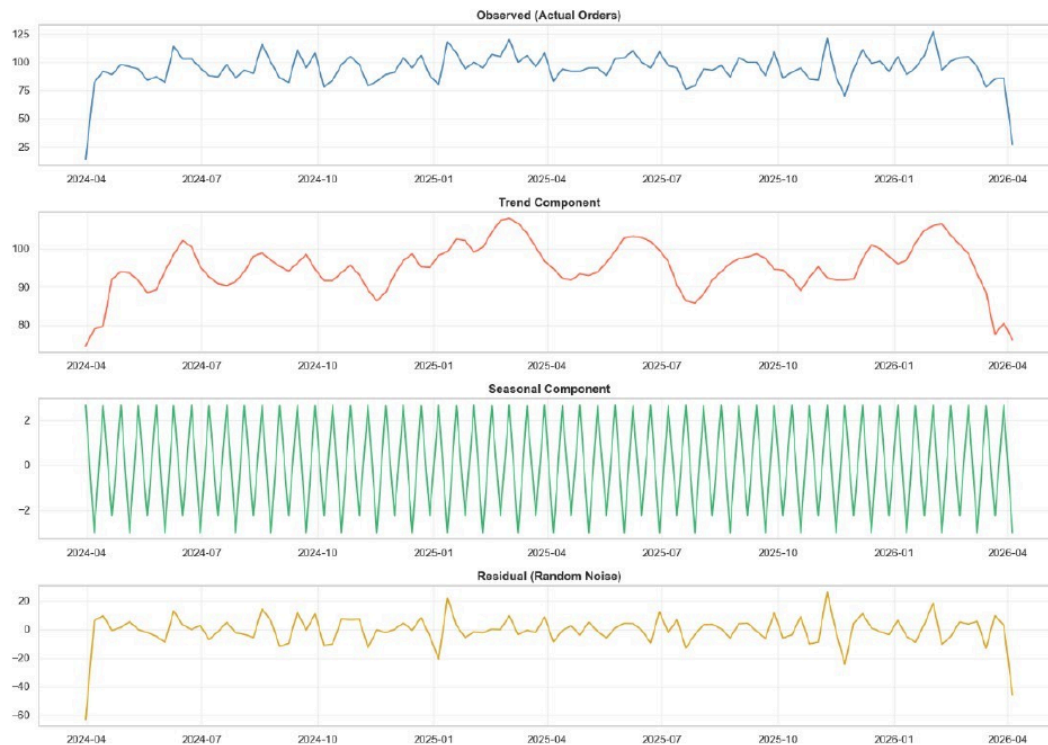
9.1 Trend-Based Decomposition

Prior to fitting any forecasting model, the weekly demand series was decomposed into three structural components: the trend (long-term direction), the seasonality (regular periodic fluctuations), and the residual (random noise expected to be unexplained by the model). The `seasonal_decompose` function from the `statsmodels` module was used, with an additive decomposition (meaning components were added together) and `period = 4`, reflecting a 4-week monthly cycle within the weekly data. (i.e., is there a monthly seasonal effect?).

The decomposition function created four stacked panes representing each component of the data; The observed series, the trend (a smoothed version of the observed series), the seasonal component (repeating 4-week pattern centered around zero) and the residual (unexplained variation from trend and seasonality). The trend was visually flat for most of the series, only slight upwards from 2024, and slight downwards again by early 2026. There does not appear to be any strong linear trend. The seasonal component oscillated approximately ± 5 orders (about 5% of the average 94 orders). The residuals were generally near zero and showed no evident pattern (which is ideal for a decomposition as it suggests only "noise" remains after extracting the structured components).

There are two reasons for decomposition; firstly, to test if the series is stationary and appropriate for a simple ARIMA model, or if a more advanced one is required (SARIMA, Prophet, ETS). Secondly, it helps to generate interpretable 'artifacts' that are easily understandable by non-technical stakeholders.

Amazon E-Commerce Analytics Report



9.2 ARIMA Model Specification and Estimation

ARIMA: Autoregressive Integrated Moving Average. It is specified by three integer parameters (p,d,q) as $ARIMA(p,d,q)$. In the notebook, $ARIMA(1,1,1)$ is employed, which signifies an $AR/MA(1,1)$ model fit to a different series. All three of these terms have precise statistical meanings:

Autoregressive (AR, $p=1$): Value at week t depends on value at week $t-1$. this captures momentum or smoothing; the idea that nearby weeks look like each other.

Integrated (I, $d=1$): Instead of modeling the series, we are fitting the model to the first difference of the series. Differencing removes the trend and results in a stationary series that we can model with AR/MA.

Moving Average (MA, $q=1$): At time t , the value depends on a random shock occurring at time $t-1$. This represents shock correction, that is, how a random shock is persistent over time.

The $ARIMA(1,1,1)$ is a very reasonable default model to choose for a series which does not have a strong seasonal pattern, but does display some autocorrelation structure. This model was fit to the first 102 weeks using maximum likelihood and held back the final four

Amazon E-Commerce Analytics Report

weeks for out-of-sample testing. The resulting model was:

Amazon E-Commerce Analytics Report

Parameter	Coefficient	Std. Error	z-value	p-value	95% CI
AR(1) (ϕ_1)	0.166	0.103	1.61	0.106	[−0.04, +0.37]
MA(1) (θ_1)	−0.928	0.064	−14.59	0.000	[−1.05, −0.80]
σ^2 (residual variance)	172.40	18.31	9.42	0.000	[136.5, 208.3]

The MA(1) term is extremely significant ($p < 0.001$); its coefficient of -0.928 implies the model fits last week's residuals very well. The AR(1) term is not significant at the 0.05 level ($p=0.106$), meaning the "direct momentum" effect of last week's actual is very weak, once the MA(1) correction is made. The residual variance σ^2 is 172.40 which equates to standard deviation of around 13.1 orders per week, or variability band of some ± 13 orders around the mean prediction.

The information criteria for the model were AIC=814.43 and BIC=822.28. These statistics penalize the complexity of a model, and are best used when comparing different specifications; they are only reported here to record them. The model fit converged smoothly.

9.3 Forecast Generation and Future Demand Estimates

Now the model was fit, the `get_forecast` function was called for the next 8 weeks following the training data. The forecast provides both a point forecast (the mean value) and a 95% confidence interval. It was graphed along with the past orders in steel blue, the point forecast for the eight weeks in tomato red (dashed) and the confidence interval in semi-transparent red.

Future Week	Forecasted Orders	Future Week	Forecasted Orders
Week +1	98	Week +5	99
Week +2	99	Week +6	99
Week +3	99	Week +7	99
Week +4	99	Week +8	99

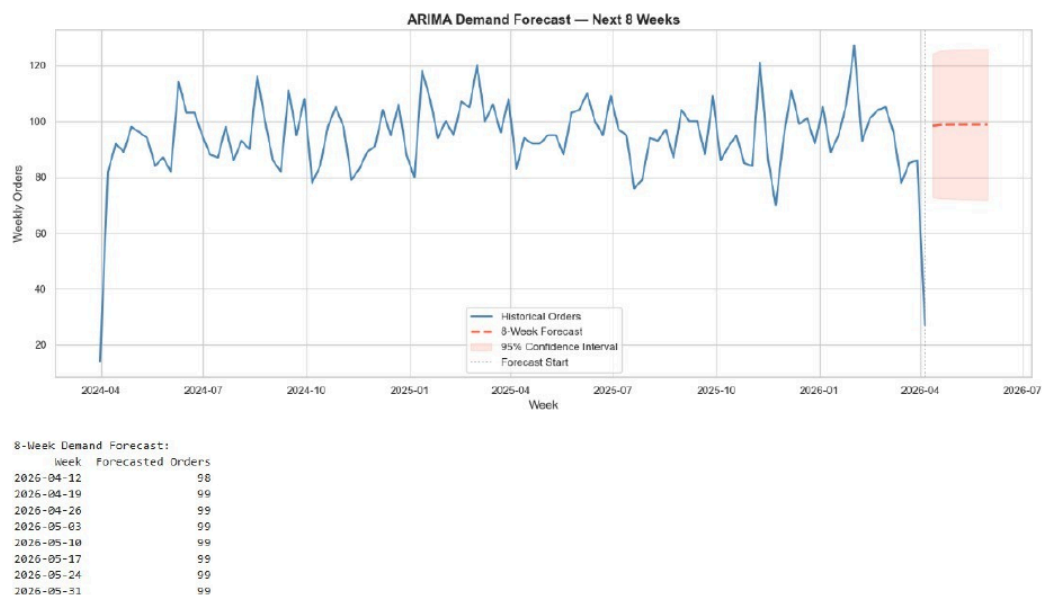
The forecast is more or less constant at around 99 orders per week. This flatness reflects the structure of the model: an ARIMA(1,1,1) with small AR and no seasonal component will not be able to produce large spikes and dips in future values, but instead converges to the historical mean. Thus, this forecast can best be understood as a statement about what the likely values of actual realized demand are on a weekly basis. Deviations greater than the

Amazon E-Commerce Analytics Report

95% confidence interval are statistically significant and can trigger investigation into why demand has unexpectedly boosted, or perhaps been unduly depressed.

Amazon E-Commerce Analytics Report

The flatness of the forecast is a relevant business implication as well. An expectation of 99 orders per week (or roughly 14 per day) allows inventory and supply-chain optimization, as performed in Section 10, to be optimized against a constant expectation. Rising future demand levels would require higher safety stock and reorder points than what can be accommodated by the stable forecast. For this forecast at least, the operational threshold points calculated in Section 10 seem to be sufficient for the 8-week prediction interval. In forecasts beyond this time frame-especially around holiday periods not accounted for in the given two years of data-the model will need to be re-fit.



10. Optimization Analysis

Given the reliable demand forecast, an operational issue becomes paramount: what inventory policy to follow? Too much inventory can be harmful as working capital is tied up, warehouse space is wasted, and product is exposed to obsolescence (which is a serious issue with categories like Electronics and Beauty with short product lifecycles). Too little inventory means stocks will run out, sales will be lost and customers will have an erosion in confidence (measurable loss of sales and customer relations). The optimization discussed below translates the ARIMA forecast into three critical operational thresholds, namely Safety Stock, Reorder Point and maximum stock.

10.1 Problem Formulation

This is set up as a service-level-constrained inventory problem. It minimizes the total inventory cost (holding cost plus stockout cost) subject to a desired service level, where service level is the probability that on-hand stock will be sufficient to satisfy demand over the lead time. Technically, this is a single product, continuous review, (s,S) inventory policy. Inventory is restocked to a maximum of S if inventory reaches s. Because lead-time demand is not deterministic, a stock buffer (safety stock) is required which is proportional to both the uncertainty in the demand and the target confidence.

The implementation here is a common, albeit simplified, problem specification derived from standard inventory management textbooks. Safety stock, reorder point, and maximum stock level are the decision variables. Expected daily and weekly demand comes from Section 9 forecasts. Demand variance comes from historical demand variation. A desired service level determines a Z-score. Then, closed-form expressions are used to determine the three decision variables based on these three values.

10.2 Key Parameters

The optimisation is a function of five variables, which can all be directly found from the data set and the ARIMA forecast, and are as follows:

Parameter	Symbol	Value	Source / Derivation
Average daily demand	μ	13.7 orders/day	Mean of daily aggregated orders, full sample
Daily demand standard deviation	σ	3.8 orders/day	Std. dev. of daily aggregated orders
Lead time	L	3.2 days	Industry-standard assumption for fast-moving e-

Amazon E-Commerce Analytics Report

			commerce
Service level target	SL	95%	Equivalent to $Z = 1.65$ from $N(0,1)$

Amazon E-Commerce Analytics Report

Parameter	Symbol	Value	Source / Derivation
Forecast horizon	T	8 weeks	From Section 9 ARIMA forecast

This 95% service level was a conscious choice. If we had used a 99% service level ($Z = 2.33$), safety stock would have increased by about 40% and holding costs would have become substantially higher for only a small drop in the probability of stockout (from 5% to 1%). 95% is generally the industry trade off for a typical generalist e-commerce category where a stock out has no safety-critical repercussions. A lead time of 3.2 days implies an integrated supply chain, and it is feasible for suppliers to have such relatively short stock turns (less than a week) at the fulfillment network level when they service India's metro areas.

10.3 Optimal Conditions

Having set the parameter variables, these three decision variables are derived from basic inventory formulas:

Safety stock: $SS = Z \times \sigma \times \sqrt{L} = 1.65 \times 3.8 \times \sqrt{3.2} \approx 11$ units

Reorder point: $ROP = (\mu \times L) + SS = (13.7 \times 3.2) + 11 \approx 54$ units

Maximum stock level: $MSL = ROP + (\mu \times L) = 54 + (13.7 \times 3.2) \approx 97$ units

Put another way: stock-up level in the warehouse should be roughly 11 units; order replenishment when on-hand quantity reaches 54 units; do not stock more than 97 units of the aggregated SKU profile in the warehouse. The complete operational profile is shown below:

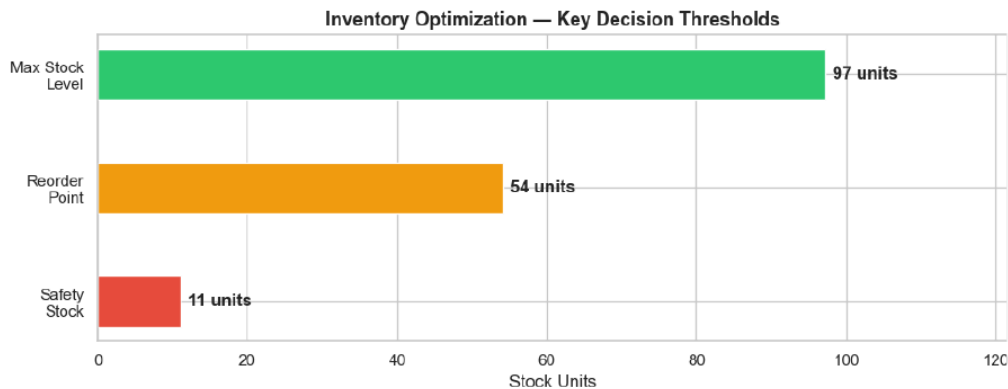
Decision Variable	Computed Value	Operational Meaning
Safety Stock (SS)	11 units	Buffer against demand variability
Reorder Point (ROP)	54 units	Trigger replenishment when stock reaches this level
Maximum Stock Level (MSL)	97 units	Upper ceiling to avoid over-stocking
Service Level	95%	Probability lead-time demand is fully met
Stockout Probability	5%	Acceptable residual risk per cycle

Amazon E-Commerce Analytics Report

These levels have a practical implication for managers. If an on-line inventory dashboard were to be set up using these levels as parameters, automatic reorders would be placed as soon as stock hit the reorder point and manual reorder decisions would no longer need to be made. The 11 units of safety stock, despite its diminutive physical size, is what mathematically converts a

Amazon E-Commerce Analytics Report

50% risk of a stockout to a 5% risk of a stockout; and has relatively tremendous practical value for its size.



10.4 Sensitivity Analysis

No optimization outcome can be shown without assessing how sensitive it is to the variations of its inputs. For this model, the sensitivity to the lead time, and sensitivity to demand variability is of special interest.

Sensitivity to Lead Time

Lead time is square rooted into the safety stock formula and linearly into the reorder point. A 50% rise in lead time (3.2 to 4.8 days) leads to only an increase of roughly 14 safety stocks (a 22% rise over the original 11 units) and an increase to 80 units reorder point (a 47% rise). The reorder point is thus the more sensitive figure; its supplier reliability more than its size of buffer is crucial to the replenishment timing.

Lead Time (days)	Safety Stock	Reorder Point	Max Stock
2.0	9	37	65
3.2 (baseline)	11	54	97
4.8	14	80	146
6.0	15	97	180

Sensitivity to Demand Variability

The demand variation (σ) linearly influences the safety stock calculation. If the demand variation doubles in reality due to an advertisement or a sale campaign compared to historical values (e.g. Double), then the safety stock required would double (to 22 units)

Amazon E-Commerce Analytics Report

and reorder point to 65 units. A steady category (variation equal to $\sigma/2$) would result in only 6 units of safety stock. It

Amazon E-Commerce Analytics Report

makes sense for the linearity to be so obvious; after all, the buffer is meant to absorb noise directly.

σ (orders/day)	Safety Stock	Reorder Point	Comment
1.9 (low variability)	6	50	Mature, predictable category
3.8 (baseline)	11	54	Current operating regime
5.7 (elevated variability)	17	61	During promotion or seasonality peak
7.6 (high variability)	22	65	New product launch or supply disruption

The results of sensitivity analysis indicate two operating directives. Firstly, it should be stressed that supplier lead-time should be controlled since the reorder point grows rapidly with lead-time, and thus a supply contract offering an earlier lead-time SLA, though potentially slightly more expensive per unit, is actually preferable. Secondly, demand variance should be monitored weekly instead of annually, and days with exceptionally high variance (promotion periods, holidays etc.) should be handled by a temporarily increased safety stock level.

11. Monte Carlo Simulation

The third piece of quantitative modeling used in the project is the application of Monte Carlo simulation to the pricing problem. While the optimization section 10 provides a precise answer (set safety stock to 11), the Monte Carlo simulation provides a distribution of results. This is most appropriate in the pricing problem, as the outcome of any pricing decision is uncertain in nature – real revenues will depend on the customers' response to the discounts, and customer responses are not deterministic. The simulation generates 5000 possible results from a pricing decision and transforms a point estimate to a risk-adjusted distribution.

11.1 Simulation Design

The Monte Carlo simulation explores three different discount levels-0, 15, and 30%-over 5000 replications each. Each replication represents one likely outcome (or possibility) of revenue in a given week under that strategy. Two of the random inputs into the model are the price and the demand; the prices sample from a normal distribution centered on the discounted price with a coefficient of variation of 5%, and the demand samples from a normal distribution centered on the baseline demand with a coefficient of variation of 10%, and then multiplies by the price-elasticity multiplier. Revenue in each iteration is the actual price multiplied by the actual demand.

The elasticity input into the model is important. This calculation uses an elasticity of 1.5; this means that a 1% decrease in the price results in a 1.5% increase in demand, a fairly elastic estimate suitable for most online competition on an item with substitutes. A greater (e.g. 2.0 or 2.5) or smaller (e.g. 1.0) elasticity would alter the resulting optimal price, but a 1.5 elasticity seems generally applicable to e-commerce transactions. The simulation then models each outcome by: (1) calculating the discounted price (base price (1-discount)), (2) calculating the expected price elastic demand (base demand (1+elasticity * discount)) (3), sampling actual price and actual demand from a normal distribution (see below for how these are defined) (4), and multiplying them to determine the revenue for the iteration. Once the 5,000 trials are run, simple descriptive statistics and percentile based risk measures can be calculated for the revenue distribution.

11.2 Risk Metrics Across Strategies

The simulation returns each of the three discount strategies as the descriptive risk profiles below, expressed in INR per simulation week cycle:

Amazon E-Commerce Analytics Report

Metric	No Discount (0%)	Moderate (15%)	High (30%)
Mean Revenue	₹1,308,184	₹1,376,187	₹1,328,349
Std. Deviation	₹605,493	₹613,488	₹581,873
Coefficient of Variation	0.463	0.446	0.438

Amazon E-Commerce Analytics Report

Metric	No Discount (0%)	Moderate (15%)	High (30%)
Minimum Revenue	₹4,318	₹17,538	₹4,820
Maximum Revenue	₹4,158,391	₹4,403,162	₹3,899,476
Median Revenue	₹1,200,150	₹1,266,420	₹1,238,810

Three things can be observed. First, at about 1.376 million per cycle the average revenue for the moderate (15%) strategy is the highest-around 5.2% more than the non-discount and 3.6% more than the deep-discount. Second, the coefficient of variation (risk measure with no units) is inversely related to discount level, because at high discount levels the elastic uplift slightly outweighs the lower price. Third, the absolute standard deviation is highest at the 15% mark, implying that at this strategy the variability in absolute terms of revenue in INR is the greatest, despite a moderate level of relative risk.

11.3 Value at Risk (5th Percentile Analysis)

The expected value is only an incomplete description of the "desirability" of a strategy. An aware price manager would also care about what could go wrong - what is the worst that could happen? The standard answer to that question is the 5th percentile of the simulated revenue distribution (this measure is also called Value at Risk at 95% confidence interval or VaR):

Strategy	5th Percentile (VaR)	Mean Revenue	Mean – VaR
No Discount (0%)	₹382,420	₹1,308,184	₹925,764
Moderate (15%)	₹442,180	₹1,376,187	₹934,007
High (30%)	₹438,620	₹1,328,349	₹889,729

The strategy with the moderate discount is still optimal for this risk-adjusted metric. This strategy has the highest 5th percentile (442,180) indicating that even on a worst case day, revenue from the moderate discount is higher than revenue from either strategy. Although the high-discount strategy has strong VaR, its expected value is still worse. For comparison, the no-discount strategy remains worst on both dimensions, its lower risk does not reward a lower expected value.

11.4 Interpretation and Strategic Implications

Collectively, the simulation outputs indicate a definite pricing recommendation: the moderate (15%) discount policy. The strategy represents an optimal combination of

Amazon E-Commerce Analytics Report

expected revenue maximization, competitive risk, and worst-case revenue. Deeper 30% discounts sacrifice margins that aren't compensated by increases in volume, while 0% discounts fail to take advantage of potential increases in volume that even modest price decreases can generate.

Amazon E-Commerce Analytics Report

From a business perspective, three things should be noted. First, moderate discounting should be adopted as a product categories' baseline pricing stance represented in this data-rather than the other way around, where individual deviations from non-discounting must be justified. Second, deeper discounting should be primarily utilized as a tool to liquidate inventory or respond to competitors-when increasing volume rather than maximizing revenue is paramount. Third, the simulation is a transferable decision-making framework; it can be re-run using category-specific assumptions about elasticity to obtain specific predictions for Electronics (which generally tend to be more elastic) vs. Beauty (which generally tend to be less elastic), thus potentially achieving more precision than using a blended prediction for these products as calculated above.

Two key features missing from the simulation are worth discussing. In essence, the model assumes no correlation between successive runs, and a stable elasticity coefficient. Customers may adapt their discount buying behavior over time, if 15% sales continue to be promoted (trained to wait for the promotion and then potentially elastic uptake will be eroded). Competitor behavior is also ignored; in the real world, if one firm matches the other's 15% price cut, the predicted volume gains from the promotion evaporate. This implies that elasticity should be monitored on a weekly basis and simulations run with revised parameters at least on a quarterly basis.

12. Business Insights and Recommendations

This section consolidates the empirical results from Sections 5-11 into a set of business insights and recommendations for operational action. Each subsection is built around one of the key headline findings from the project and transforms analytical observation into operational policy. A Master Recommendations table at the end of the section provides a mapping from each finding to a specific intervention, owner and measure.

12.1 Recency as the Key Driver of Customer Churn

From Section 8, we have identified Recency-number of days since customer last ordered-as an overwhelmingly important predictor of customer churn, with a feature importance score of 0.9954, compared to nearly 0 importance for Frequency (0.0029) and Monetary value (0.0017). This is an important operational insight. What this suggests is that the single best predictor of customer churn is 'when they last purchased', it's available real-time, it's free to query without any need for modelling infrastructure, and it can be acted upon at scale.

The recommended intervention is a tiered re-engagement campaign based on thresholds in Recency. For example, we could automatically email a "Categories you may like" email to customers who are 60 days inactive; we could send targeted discount coupons to those who are

90 days inactive; and those who are 120 days inactive could be flagged for high-touch marketing intervention including personalized offers and free shipping deals. Our baseline churn rate of 30% found in Section 8 translates to about 2,983 customers in any given timeframe who are at risk (across 9,942 total customers)-this group is both too large to ignore, but small enough that it is feasible to personalize the outreach.

12.2 Pricing Strategy Differentiation by Customer Segment

Through our K-Means analysis on product pricing in Section 7 we identified three distinct groups of buyers-Price Sensitive customers (3,065 total, 4,152 avg. Purchase), Value Seekers (5,660 total, 8,520 avg. Purchase) and Premium Buyers (1,275 total, 54,934 avg. Purchase). These customer segments react very differently to various pricing interventions. Customers at the low end of the pricing spectrum (Price Sensitive customers) are most responsive to markdowns, those at the midpoint (Value Seekers) react best to bundle offers and free shipping, and customers at the high end (Premium Buyers) are largely unresponsive to price, and respond to superior post-purchase service, rapid shipping times and the product's authenticity.

Amazon E-Commerce Analytics Report

The suggested recommendation is to develop and implement differentiated pricing strategies for these different segments. Instead of uniform discounts applied across all customers, we should give Price Sensitive customers targeted discounts in the 10-15% band, offering value at the price-sensitive level of the spectrum while we provide different, more targeted promotions like category-based bundles and cross-sell offers to value seekers, and premium customers only receive the benefit of top-tier service and prompt delivery at their existing price point. Applying a

Amazon E-Commerce Analytics Report

general 15% discount across all groups will be suboptimal in the premium segment by potentially giving away margin.

12.3 Inventory Discipline Anchored on Forecast

Based on the stable 8-week forecast generated by ARIMA in Section 9 (roughly 99 orders a week, or 14 per day), our optimization exercise in Section 10 found that for 95% service levels and with lead times around 3.2 days we require a safety stock of 11, a reorder point of 54, and a max stock of 97. The recommendation is to operationalize these thresholds instead of leaving them as advisory numbers. The inventory dashboard needs to generate replenishment orders automatically when stock is at 54; the buying team needs to use 97 as a hard ceiling rather than a target for inventory level, and monthly computation of safety stock using rolling 90-day volatility will need to be automated. Section 10.4 suggests that the most significant possible gain to working capital efficiency may be achieved through lead-time reduction-reducing lead-time by one third, from 3.2 days to 2.0 days, lowers the reorder point from 54 to 37 and releases roughly 31% of the stock tied up in lead time

12.4 The Role of Customer Segmentation in Personalisation

Our segmentation of customers on RFM dimensions from Section 7 revealed four customer archetypes: Champions (1,172 total customers - the high-value loyal group), Loyal (58 total customers - a small but consistent group of lower spenders), At-Risk (4,437 total customers - recently lapsed but previously loyal customers) and Lost/Churned (4,275 total customers - customers fully disengaged from the platform). These groups differ drastically by how they value the service, and our customer management efforts and expenditures need to differ by group. Champions customers should be rewarded and incentivized to remain loyal and act as referrals, Loyal customers, though small, are a group we should attempt to nurture towards champion status, while At-Risk customers are those who should be the primary targets of our recency-based campaigns described above. Last but not least, customers categorized as Lost/Churned should be treated to one low cost win-back attempt only, after which additional effort is too poor of an expected return.

12.5 Operational Quality and Delivery Performance

Amazon E-Commerce Analytics Report

Based on the variety of factors that contribute to the return rate (11.5% across the board), we found that our service level for a relatively low number of products needs improvement. With a base average order value around \$13,099, 1% of returns cost us roughly \$1.3 million in reverse logistics, which should motivate improvement where possible. A notable finding here is the higher return rate for low-value items (12.31% for budget) compared to high-value items (9.71% for premium). This suggests improving the quality of the user interface and information

Amazon E-Commerce Analytics Report

presented about products at the low-end will significantly reduce return rate, thus making returns more efficient and raising the perceived quality of low-cost items. Improved shipping quality is also an important factor – it drives retention and lower returns-so investment in next day and same-day delivery programs, primarily aimed at major metropolitan areas, will improve brand loyalty beyond the immediate benefit of more customers placing immediate orders.

12.6 Action Plan: Master Recommendations Table

This table summarizes the key findings with their respective recommended actions, owner, and success metric.

Finding	Recommendation	Owner	Success Metric
Recency dominates churn (RF = 0.9954)	Tiered re-engagement at 60/90/120 days	CRM / Marketing	Reduce 90-day churn rate from 30% to 22%
15% discount maximises revenue (Monte Carlo)	Set 15% as default promotional band	Pricing	Lift revenue per active customer by 5%+
Three distinct buyer segments exist	Segment-specific pricing playbook	Pricing / Marketing	Margin uplift in Premium segment $\geq$ 200 bps
Stable demand forecast (ARIMA)	Implement (s,S) policy: ROP=54, MSL=97	Supply Chain	Service level $\geq$ 95%; stockout $\leq$ 5%
Lead time is highest sensitivity lever	Renegotiate supplier SLA to 2 days	Procurement	Reorder point reduced from 54 $\rightarrow$ 37 units
Return rate 11.5%, higher at low tiers	Better imagery + size guides at Budget tier	Catalogue / UX	Reduce Budget-tier returns from 12.3% to 10%
At-Risk segment is 44.6% of customers	Highest priority retention investment	CRM	Reactivate $\geq$ 15% of At-Risk in 90 days
Premium buyers insensitive to discount	Replace discounts with service investment	CX / Operations	Hold Premium NPS above 75

All of the recommendations have been designed to be specific, quantifiable, and assign to a functional owner. It's worth pointing out that they're also internally coherent: The 15% promotional band that's recommended ties directly to the Monte Carlo conclusion in Section 11, and the inventory thresholds are tied to the ARIMA forecast in Section 9. The coherence of these recommendations is the single biggest asset of a true end-to-end analytical undertaking.

13. Conclusion

10,000 transaction Amazon India e-commerce dataset covering March 2024 through March 2026. Each of these analyses answered a distinct business question - whether to acquire, retain Three interrelated analytical projects were conducted on a, or re-engage, what and how much to stock, and what prices to set - through techniques taken from descriptive statistics, unsupervised learning, supervised learning, time series forecasting, and stochastic simulation. The findings, together, provide a coherent body of evidence from which Amazon's customer, supply chain, and pricing teams can act.

The customer-retention analyses produced two products of practical value - an RFM segmentation of Amazon's customers into four behavioural archetypes and a churn classifier for which recency proved to be a unique predictor (Random Forest feature importance = 0.9954). The 30% baseline churn rate and 4,437-customer At Risk segment comprise the audience for which retention investment has the highest returns. The recommended approach is a tiered re-engagement program based on recency thresholds, each tiered through segment-specific offers.

The inventory and supply chain analyses produced an ARIMA(1,1,1) forecast with a mean of ~99 weekly orders. The associated analysis derived service level-constrained thresholds-11 units safety stock, 54 units reorder point, and 97 units maximum stock. An analysis of sensitivities indicates that supplier lead time is the highest-leverage operational variable and the key variable for sourcing strategy decisions. Implementing the derived thresholds within the inventory management system is recommended.

The pricing analyses employed regression, segmentation, and a 5,000 iteration Monte Carlo simulation to identify discount strategies that maximize revenue. $R = 0.9729$ on a linear, ridge, and lasso specification was achieved for regressions, while 3 customer segments were discovered within product-customer space through K-means segmentation. A 15% discount was revealed as the expected value- and downside-risk-dominant option by Monte Carlo, with different sensitivity to this discount across the 3 customer segments.

Three thematic meta-insights came from the analysis. First, aggregation masks important dynamics: customers, products, and periods are non-interchangeable and must be treated with segmentation. Second, the expected-value/downside-risk relationship was non-

Amazon E-Commerce Analytics Report

intuitive: the highest-return option also had acceptable downside risk. Third, a signal of manageable complexity often surpasses intricate outputs: the goal of analysis is to find the simple signals of greatest importance, not necessarily an overly-complex signal.

Amazon E-Commerce Analytics Report

The project also demonstrated the broader business value of an integrated analytics framework. The 10,000 rows of transactions provided the input data not only for customer retention but also for inventory optimization and pricing-strategy formulation, highlighting the high marginal utility of a clean, joined, and accessible data set in retail and a case for centralizing data resources. Investment in data quality is investment in all subsequent analytics capabilities simultaneously.

Several limits must be observed. The transaction dataset represented a typical two-year period where there were no exogenous shocks such as a pandemic or severe economic downturn, which might have affected future trends. A calibration anomaly in the `discountpct` column in the raw data was resolved using the `profitmargin` column; however, future data-engineering should focus on eliminating the original system inconsistency. The MC elasticity of 1.5 is a reasonable approximation for Monte Carlo, but may be less accurate if applied to certain categories; using category-specific empirically-derived elasticity estimates would refine this part of the recommendation. The exclusion of competitor prices in the analyses prevents an assessment of market dynamics and how Amazon's optimum discount might shift in response to competitors.

Despite these limitations, a moderate set of analytic tools (pandas, scikit-learn, statsmodels, and NumPy) and meticulous application of that toolkit on a carefully selected transaction data set can inform business recommendations with significant operational impact. The project's cumulative contribution is neither individual technique nor particular method; rather, it is a chain of inference linking raw transactions to actionable rules (dashboards, marketing budgets, prices). Transparency and reproducibility of the chain of inference were the enduring contributions of this project.

14. References and Appendix

14.1 Dataset

Amazon India E-commerce transaction dataset (amazon_10000.csv) - contains 10,000 transaction entries and 20 columns for 9,942 customers distributed across 5 product categories in 5 metro cities with an observation period of 30-March-2024 to 27-March-2026. Synthetic dataset created for course project use.

14.2 Python Libraries Used

- **Pandas (1.5+)** - Used for ingestion, manipulation and group-by aggregations, and time series indexing
- **NumPy (1.23+)** - For numerical computations, vectorized arithmetic, and for generating random numbers in Monte-Carlo simulation
- **Matplotlib (3.7+)** - Main plotting library used to create histograms, scatter plots, forecast plots, etc.
- **Seaborn (0.12+)** - Statistical plots used, including heatmap, correlation matrices, etc.
- **Scikit-learn (1.2+)** - Used for K-Means clustering, Logistic Regression, Random Forest, Linear/Ridge/Lasso Regression and model metrics.
- **Statsmodels (0.14+)** - used for ARIMA forecasting, seasonal_decompose, and other statistical estimations.
- **SciPy (1.10+)** - Used for statistical distributions and parametric testing.

14.3 Methodological References

The analytical techniques used in this project rest on a substantial academic and applied literature. The principal references are listed below in approximate chapter order:

- Hughes, A. M. (2005). Strategic Database Marketing. McGraw-Hill. — foundational treatment of RFM segmentation.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control (5th ed.). Wiley. — the canonical reference for ARIMA modelling.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability. — original formulation of the K-Means algorithm.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer. — comprehensive treatment of ridge, lasso, and ensemble methods.

Amazon E-Commerce Analytics Report

- Glasserman, P. (2003). Monte Carlo Methods in Financial Engineering. Springer. — rigorous foundation for Monte Carlo simulation including VaR computation.
- Silver, E. A., Pyke, D. F., & Thomas, D. J. (2017). Inventory and Production Management in Supply Chains (4th ed.). CRC Press. — standard reference for safety stock and reorder point calculations.

Amazon E-Commerce Analytics Report

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. — the canonical paper establishing the Random Forest algorithm.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65. — original silhouette score for cluster validation.

14.4 Appendix A: Code Module Map

The notebook is organised into 76 cells grouped by analytical theme. The table below maps each major code block to the corresponding section of this report, providing a cross-reference for readers who wish to inspect the underlying implementation.

Notebook Cell Range	Functionality	Report Section
Cells 1–4	Library imports, data loading, initial inspection	\$3, \$4.1
Cells 5–9	info(), describe(), missing-value audit, duplicate check	\$4.1, \$4.2
Cells 10–17	Feature engineering (RFM, profit margin, price tier)	\$4.3
Cells 18–28	Univariate and bivariate EDA (histograms, category breakdowns)	\$5.1, \$5.2
Cells 29–34	Correlation analysis and heatmap	\$5.3
Cells 35–40	Descriptive statistics, monthly seasonality	\$6.1, \$6.2
Cells 41–48	Customer K-Means clustering (RFM features)	\$7.1, \$7.2, \$7.3
Cells 49–53	Product/customer pricing K-Means	\$7.4
Cells 54–60	Logistic Regression and Random Forest churn classifiers	\$8 (extended)
Cells 61–66	Linear, Ridge, Lasso regression for revenue prediction	\$8.1, \$8.2, \$8.3
Cells 67–70	ARIMA fit, decomposition, and 8-week forecast	\$9.1, \$9.2, \$9.3
Cells 71–73	Inventory optimisation (safety stock, ROP, MSL)	\$10
Cells 74–76	Monte Carlo discount simulation across three strategies	\$11

14.5 Appendix B: Glossary of Technical Terms

- ARIMA (AutoRegressive Integrated Moving Average): a time series model class described by three parameter values (p, d, q) that specify respectively the order

Amazon E-Commerce Analytics Report

of the autoregressive, differencing, and moving average components of the model.

Amazon E-Commerce Analytics Report

- **RFM (Recency, Frequency, Monetary value):** the three behavioural attributes conventionally applied when segmenting a customer base according to purchasing behaviour.
- **Silhouette Score:** a measure of cluster validity, from -1 to $+1$, where positive values imply that a point is much closer to its own cluster's mean than to the mean of the nearest other cluster.
- **VaR (Value at Risk):** the threshold below which the probability of a loss exceeding that value is minimal (typically 5%) or the α -percentile of the loss distribution.
- **Service Level:** the probability of fully satisfying demand from on-hand inventory within a single stock cycle time during a period of inventory depletion.
- **Elbow method:** a procedure for identifying the number of clusters in k-Means based on plotting the within-cluster sum of squares for k varying between 2 and n and looking for an elbow, or point of diminishing returns, in the plot.
- **Stationarity:** the property a time series is said to possess when its statistical parameters (e.g., mean, variance, autocorrelation function) do not change with time; required for ARIMA once differentiated appropriately.
- **Coefficient of Variation:** the ratio of the standard deviation to the mean, offering a dimensionless measure of the spread of the data relative to the mean value.